Go основы

Источники:

- METANIT
- Go Tour
- Google Go Open Source

Оглавление

- Программы и пакеты
  - Импорт пакетов
  - GOROOT
  - GOPATH
  - Пакеты, поиск
  - vendor
  - Режимы сборки
- GO ENV
- Основные команды go
  - go run
  - go build
  - go install
  - go test
  - go mod init
  - go mod tidy
  - go get
  - go mod vendor
  - go env
  - go fmt
  - gofmt
  - go vet
  - go clean
  - go list
  - go doc
  - go version
  - go work

    - go generate
    - go tool
    - ШПОРА
- Типы данных
- Функции
- Переменные
- Константы
- Приведение типов
- Циклы
- Условные конструкции
- Defer
- Замыкания
- Указатели
- Структуры
- Методы структур
    - Как выбрать ptr или по значению?
- Массивы и слайсы
    - Добавление элементов в слайс
    - Удаление элементов из слайса
    - Копирование слайса
    - Сортировка слайса
    - Сравнение слайсов, пакет reflect
    - Инициализация слайса
    - !!! ОЧЕНЬ ВАЖНО !!!
- Итерирование
- Map
    - Вставка и обновление
    - Получение значения по ключу
    - Перебор мапы
    - Важно!!!
    - Concurrency
- Интерфейсы
    - Empty interface
    - Stringer
    - Еще про интерфейсы
- Errors
- Reader
- Generics и Параметры типа

    - Параметры типа в функции
    - Constraint для generics
    - Generic структура
    - Generic интерфейсы
    - Разница generic и interface
- Горутины
    - Контекст горутин
    - Синтаксис горутин
    - Sleep
    - Синхронизация горутин и WaitGroup
- Каналы
    - Оператор канала <-
    - Небуферизованные каналы
    - Обмен данных через каналы
    - Буферизованные каналы
    - Возвращение канала
    - Закрытие канала
    - Двунаправленные и однонаправленные каналы
    - Синхронизация горутин через каналы
    - Получение значений канала в цикле
    - Подходы использования каналов
- Гонка данных и обнаружение
- Mutex
- RWMutex
- Обработка ошибок
    - errors.New()
    - Wrapping
    - Unwrapping
    - errors.Is() и errors.As()
    - panic(), defer, recover()
- Контекст
    - context.Background
    - context.TODO
    - context.WithCancel
    - context.WithTimeout
    - context.WithDeadline
    - context.WithValue
    - context.Done

    - context.Err
    - context.Deadline
    - context.Value
    - context.WithCancelCause
    - context.Cause
    - context.AfterFunc
    - Context SHPORA
- Garbage collector
    - Куча и стэк
        - Стэк
        - Куча
        - escape analysis
    - Механизм работы
    - Запуск GC
    - Задать GOGC
    - GOMEMLIMIT
    - Памятка по GC
- Scheduler
    - GMP
        - Goroutine
        - Machine
        - Processor
        - Связь GMP
        - GOMAXPROCS
    - Состояния goroutine
    - Очереди Scheduler
        - Local run queue
        - Global run queue
        - Work stealing
    - Что происходит при go f()
    - Поведение при блокировках
        - При блокировке на channel
        - При time.Sleep
        - При mutex
        - Network I/O и netpoller
        - При syscall
    - Preemption
    - Общая схема и заключение

- Строки
  - Длина строки
  - Индексация строки
  - Rune
  - Range по строке
  - Изменение строк
  - Срез строки
  - Строковые литералы
    - Interpreted string literal
    - Raw string literal
  - Сравнение строк
  - Пакет strings
  - Самые частые функции
  - String Builder

Программы и пакеты

В Go программы собираются из пакетов.

Пакеты представляют удобную организацию разделения кода на отдельные части и модули.

Физически пакет - каталог/папка, которая содержит несколько файлов с исходным кодом на Go.

При импорте пакета мы можем использовать константы, типы, функции и т.д. многократно в различных программах.

Код пакета располагается в одном или нескольких файлах с расширением .go.

Чтобы определить пакет, нужно в самом начале файла написать package имя_пакета.

Программа начинает работу из пакета main.

Пакеты бывают:

- исполняемые/executable (main)
- библиотеки/reusable

Импорт пакетов

Экспортируемые имена начинаются с заглавной буквы, не экспортируемые с маленькой. Пакеты импортируются так:

```
import "fmt"
import "math/rand"
```

или разом:

```
import(
  "fmt"
  "math/rand"
)
```

Чтобы использовать имена из пакета, нужно обращаться через последнее слово в импорте, например:

```
import "github.com/aboba/…/math/pack"

func main() {
  //…
  fmt.Println(pack.Value)
}
```

Или поставить алиас для пакета:

```
import (
  "fmt"
  myPack "github.com/…/vladlim/dormLife"
)

func main() {
  //…
  fmt.Println(myPack.RandomMethod())
}
```

Пакеты можно импортировать как собственные, так и встроенные и чужие.

Список встроенных пакетов тут.

Чтобы не обращаться к пакету через его имя, можно использовать dot import:

```
import (
  . "fmt"
  . "math"
)
```

```
func main() {
  Println(Pow(2, 3))
}
```

Но тут может возникнуть конфликт имен, если в разных пакетах есть одноименные функции или переменные, поэтому лучше это не использовать.

Если пакет импортирован, его обязательно использовать, иначе компилятор будет ругаться.

Можно дать алиас названию пакета и обращаться через него

```
import (
  auth "github.com/vladlim/dormLife/auth_service"
)

func main() {
  //…
  auth.SomeFunc()
}
```

GOROOT

GOROOT - папка с установленным Go SDK.

SDK (Software Development Kit) - набор инструментов для разработки под какую-то технологию.

Содержит обычно:

- компилятор
- стандартную библиотеку
- утилиты
- документацию
- иногда отладчики, генераторы, CLI

Go SDK содержит:

- go // главная CLI команда
- gofmt // форматтер
- compile // компилятор
- link // линкер
- стандартная библиотека
- исходники стандартной библиотеки

Например, когда пишу

```
go version
```

Я хожу к установленному Go SDK.

Обычно он лежит в

```
user/local/go
C:\Program Files\Go
```

Внутри примерно:

```
GOROOT/
  bin/
    go
    gofmt
  src/
    fmt/
    net/
    strings/
    context/
  pkg/
```

Чтобы посмотреть:

```
go env GOROOT
```

GOPATH

GOPATH - рабочая область/кеш установленных внешних пакетов Go: модули, бинарники.

```
go env GOPATH
```

Внутри:

```
GOPATH/
  bin/
    golang-lint
    gopls
    mockgen
    swag
    ...
```

```
pkg/
  mod/
    git-molva.ru
    go.yaml.in
    cache
    ...
```

Пакеты, поиск

Один пакет - один каталог.

То есть в одной папке у всех файлов должен быть один пакет (исключение-тесты).

go.mod - файл с основными зависимостями пакетов. Объявляет путь к модулю, версию Go, минимальные версии зависимостей.

go.sum - файл с контрольной суммой зависимостей. Нужен чтобы go убедился, что устанавливает нужные версии пакетов (проверяются хеши версий). Хранит все зависимости: транзитивные, прямые, старые версии и тд.

Модуль - коллекция пакетов.

Модуль обладает:

- изоляцией зависимостей: позволяет управлять своими зависимостями независимо, что позволяет избежать конфликтов версий между проектами.
- воспроизводимые сборки: файлы go.mod (конфигурация модуля) и go.sum (криптографические контрольные суммы для проверки целостности зависимостей) гарантируют, что любой, кто клонирует репу сможет собрать проект с теми же версиями зависимостей.
- упрощенное управление версиями: модули позволяют указывать минимальные требуемые версии зависимостей, использовать алиасы для пакетов, а также заменять локальными версиями для разработки.
- децентрализованная разработка: можно загружать зависимости из различных источников, например, с Git Hub.

go.sum связан с GOSUMDB - открытая БД хешей пакетов, у которой как раз спрашивают, какой хеш имеет вот эта версия пакета.

Если проект приватный, то настраивается GOPRIVATE и перенастраивается GOSUMDB.

Go при сборке смотрит на строку импорта:

```
import "fmt"
```

```
// или

import "github.com/google/uuid"
```

1. Если соответствует стандартному пакету, то берет его из GOROOT

```
import (
  "fmt"
  "strings"
  "net/http"
  "context"
  "os"
  // ...
)
```

Идет в:

```
$GOROOT/src/fmt
$GOROOT/src/net/http
```

В $GOROOT/src/ лежат стандартные пакеты:

```
usr/local/go/
  src/
    fmt/
      errors.go
      doc.go
      stringer.go
      print.go
      ...
    net/
      http/
        request.go
        transport.go
        cookie.go
        server.go
        ...
        ...
      smtp/
      url/
      ...
    ...
```

2. Если есть go.mod, то идет искать в пакетах текущего модуля:

go.mod

```
module example.com/myapp

go 1.22

//...
```

Локальные пакеты импортируются через module path:

```
import "example.com/myapp/internal/auth"
```

Допустим структура такая:

```
myapp/
  go.mod
  main.go
  internal/
    auth/
      service.go
```

Тогда:

```
import "example.com/myapp/internal/auth"
```

Означает "возьми пакет из ./internal/auth", потому что example.com/myapp совпадает с module в go.mod.

3. Потом внешние зависимости из go.mod / module cache

Если импорт такой:

```
import "github.com/google/uuid"
```

То Go идет смотреть go.mod:

```
require github.com/google/uuid v1.6.0
```

Если модуль уже скачан, берет его из кеша:

```
$GOPATH/pkg/mod
```

Вот этот pkg/mod это GOMODCACHE, где сохраняются исходники скачанных внешних пакетов.

GOMODCACHE можно настроить отдельно, но обычно просто так это не делают.

Если не скачан, то Go может скачать его при команде:

```
go mod tidy
go build
go test
go run
```

Если упрощенно:

```
import "fmt"
    -> стандартная библиотека
    -> GOROOT

import "example.com/myapp/internal/auth"
    -> локальный пакет текущего модуля
    -> папка внутри проекта

import "github.com/google/uuid"
    -> внешний модуль
    -> если vendor mode: ./vendor
    -> иначе: GOMODCACHE, а если нет — скачать
```

vendor

vendor - это папка внутри проекта, куда можно положить копии зависимостей.

Это нужно для того, чтобы брать зависимости не из GOMODCACHE или скачивать из интернета, а брать из локальной папки vendor.

Создание:

```
go mod vendor
```

Создает папку vendor/ и складывает туда зависимости, нужные проекту.

```
go mod tidy
go mod vendor
```

После этого структура может быть такая:

```
project/
  go.mod
```

```
go.sum
vendor/
  modules.txt
  github.com/
    google/
      uuid/
```

vendor/modules.txt - файл, в котором описаны модули и версии, которые лежат в vendor.

Go использует его, чтобы понять, что vendor соответствует go.mod.

Если go.mod и vendor/modules.txt расходятся -> получим ошибку.

Тогда нужно выполнить:

```
go mod vendor
```

vendor используют если хотим:

1. собирать проект без доступа к интернету
2. зафиксировать копии зависимостей внутри репозитория
3. сделать сборку более воспроизводимой
4. избежать зависимости от внешнего proxy/cache во время build
5. пройти требования компании/security/compliance

Если зависимости поменяются, вендор приходится обновлять. При этом нужно следить за соответствием go.mod и vendor.

Режимы сборки

Есть два основных режима сборки проекта:

- Через modules/GOMODCACHE:

```
go build -mod=mod
```

- Через vendor:

```
go build -mod-=vendor
```

В зависимости от режима Go будет искать внешние зависимости в кеше или вендоре.

GO ENV

go env - команда, которая показывает среду Go toolchain, то есть настройки, с которыми команда go собирает, тестирует, скачивает зависимости и запускает проекты.

```
❯ go env
AR='ar'
CC='gcc'
CGO_CFLAGS='-O2 -g'
CGO_CPPFLAGS=''
CGO_CXXFLAGS='-O2 -g'
CGO_ENABLED='1'
CGO_FFLAGS='-O2 -g'
CGO_LDFLAGS='-O2 -g'
CXX='g++'
GCCGO='gccgo'
GO111MODULE=''
GOAMD64='v1'
GOARCH='amd64'
GOAUTH='netrc'
GOBIN=''
GOCACHE='/home/vlad/.cache/go-build'
GOCACHEPROG=''
GODEBUG=''
GOENV='/home/vlad/.config/go/env'
GOEXE=''
GOEXPERIMENT=''
GOFIPS140='off'
GOFLAGS=''
GOGCCFLAGS='-fPIC -m64 -pthread -Wl,--no-gc-sections -fmessage-length=0 -ffile-prefix-map=/tmp
GOHOSTARCH='amd64'
GOHOSTOS='linux'
GOINSECURE=''
GOMOD='/usr/local/go/src/go.mod'
GOMODCACHE='/home/vlad/go/pkg/mod'
GONOPROXY=''
GONOSUMDB=''
GOOS='linux'
GOPATH='/home/vlad/go'
GOPRIVATE=''
GOPROXY='https://proxy.golang.org,direct'
GOROOT='/usr/local/go'
GOSUMDB='sum.golang.org'
GOTELEMETRY='local'
GOTELEMETRYDIR='/home/vlad/.config/go/telemetry'
GOTMPDIR=''
GOTOOLCHAIN='local'
GOTOOLDIR='/usr/local/go/pkg/tool/linux_amd64'
GOVCS=''
GOVERSION='go1.25.3'
```

```
GOWORK=''
PKG_CONFIG='pkg-config'
```

Часть берется из окружения ОС, часть из файла GOENV, часть выставляется самим Go. Настройки, записанные через go env -w, хранятся в пользовательском конфиге Go; посмотреть конкретное значение можно так:

```
go env GOPATH
```

Основные пути:

```
GOROOT      — где установлен Go
GOPATH      — рабочая область Go
GOMOD       — какой go.mod сейчас активен
GOMODCACHE  — где лежат скачанные зависимости
GOCACHE     — где лежит кэш сборки
GOOS        — под какую ОС собираем
GOARCH      — под какую архитектуру собираем
GOVERSION   — версия Go
GOPROXY     — откуда качаем зависимости
GOPRIVATE   — какие зависимости приватные
GOWORK      — активен ли go.work
GOBIN       — куда ставятся go install бинарники
GOFLAGS     — глобальные флаги go-команд
```

Если есть проблема с тем, что пакеты закешировались неправильно или что-то такого рода:

```
go clean -modcache # Чистка кеша модулей
```

Основные команды go

1. `go run`

```
go run main.go
go run .
```

Компилирует программу во временную директорию и сразу запускает ее.

Можно передавать аргументы/флаги

```
go run . arg1 arg2 # передали аргументы
go run cmd/main.go --config_path configs/config.yaml --log_level debug # передали два флага
```

В программе аргументы будут доступны через os.Args []string:

```
package main

import (
    "fmt"
    "os"
)

func main() {
    fmt.Println(os.Args)
}
// ❯ go run main.go 1 "2" --example_flag "example flag str"
// [/home/vlad/.cache/go-build/e7/e7e9e33bd1a20d7cdba6678ecac16ff86d74e2770e91fe107bb1da428bd2
// [1 2 --example_flag example flag str]
// ❯ go clean cache
// package cache is not in std (/usr/local/go/src/cache)
// ❯ go clean -cache
// ❯ go run main.go 1 "2" --example_flag "example flag str"
// [/tmp/go-build3653726347/b001/exe/main 1 2 --example_flag example flag str]
// [1 2 --example_flag example flag str]
```

При первом запуске через go run взялся ранее скомпилированный бинарь из GOCACHE. os.Args[0] это как раз адрес бинарника.

Далее почистил кеш и вызвал заново, уже новый скомпилированный бинарь положили в GOCACHE.

То есть os.Args содержит все аргументы, включая путь к программе.

flag.Args() - аргументы после обработки флагов без os.Args[0]

В примере выше все обработалось как обычные аргументы, а не как флаги, потому что я не объявил их.

При этом стандартный flag.Parse() рассчитывает, что сначала идут флаги, а потом позиционные аргументы, иначе если он сначала найдет позиционный аргумент, то перестанет искать флаги.

Исправим:

```
package main

import (
    "fmt"
    "os"
    "flag"
)

func main() {
```

```
    exampleFlag := flag.String("example_flag", "", "example flag description")
    fmt.Println("example flag: ", exampleFlag)

    fmt.Println("os.Args: ", os.Args)

    flag.Parse()
    fmt.Println("ptr: ", exampleFlag)
    fmt.Println("value: ", *exampleFlag)
    fmt.Println("flag.Args: ", flag.Args())

}
// ❯ go run main.go --example_flag 'example flag str' 1 2 3
// example flag:  0xc00002e070
// os.Args:  [/home/vlad/.cache/go-build/ac/ac73d244ffc430602a5c3ac00e021b37095430d8cc52762ca9
// ptr:  0xc00002e070
// value:  example flag str
// flag.Args:  [1 2 3]
```

2. `go build`

```
go build
go build .
go build ./... # собрать все пакеты проекта
go build -o имя_бинарника main.go # собрать main.go и сохранить бинарь имя_бинарника

# Потом go run или go run ./имя_бинарника
```

Компилирует указанные пакеты вместе с зависимостями и сохраняет бинарник.

3. `go install`

```
go install github.com/swaggo/swag/cmd/swag@latest
```

Компилирует и сохраняет бинарник внешних пакетов.

После этого бинарник будет лежать в $GOPATH/bin или $GOBIN.

То есть:

```
go build   — кладет бинарник обычно в текущую директорию
go install — кладет бинарник в GOBIN/GOPATH/bin
```

4. `go test`

```
go test ./... # запуск всех тестов
go test -v ./... # подробный вывод
go test -run TestName ./... # запуск конкретного теста
```

```
go test -race ./... # проверка race condition
go test -cover ./... # покрытие
go test -coverprofile=coverage.out ./... # сохранить coverage в файл
go tool cover -html=coverage.out # просмотр coverage в браузере
```

Запускает тесты. При запуске Go ищет файлы *_test.go и тестовые функции вида:

```
func TextXxx(t *testing.T)
```

Работает вместе с пакетом testing, который поддерживает автоматическое тестирование пакетов.

5. `go mod init`

```
go mod init example.com/myapp
```

Создает go.mod файл, который отслеживает зависимости модуля:

```
module example.com/myapp

go 1.xx

require example.com/utils v1.x.x
#...
```

6. `go mod tidy`

```
go mod tidy
```

Приводит go.mod и go.sum в соответствие исходному коду:

- добавляет недостающие зависимости
- удаляет неиспользуемые зависимости
- обновляет/создает go.sum

7. `go get`

```
go get github.com/google/uuid
```

Устанавливает зависимость (как go install).

```
go get github.com/google/uuid # добавить зависимость
go get github.com/google/uuid@v1.6.0 # указать версию
```

```
go get github.com/google/uuid@latest # обновить зависимость до последней версии
go get github.com/google/uuid@1.5.0. # откатить до версии 1.5.0
```

8. `go mod vendor`

```
go mod vendor
```

Создает папку vendor/ и складывает туда зависимости, нужные проекту.

9. `go env`

Разбирал выше. Показывает/изменяет настройки, с которыми запускается/собирается проект.

```
go env GOROOT # посмотреть настройку

go env -w GOPRIVATE=github.com/company/* # записать настройку

go env -u GOPRIVATE # сбросить настройку
```

10. `go fmt`

```
go fmt ./... # форматирует Go-код
```

Обертка над gofmt.

11. `gofmt`

```
gofmt -w .
gofmt -w ./...
```

Форматирует .go файлы.

Разница:

```
go fmt - работает по пакетам
gofmt - по файлам/директориям
```

`-w` - записать изменения в файлы. Без `-w` разультат выводится в stdout.

12. `go vet`

```
go vet ./...
```

Ищет подозрительные места в коде, например:

```
package main

import (
    "fmt"
)

func main() {
    var s string
    fmt.Printf("%d", s)
}

// ❯ go vet ./...
// # example.com/tour_test
// # [example.com/tour_test]
// ./main.go:9:14: fmt.Printf format %d has arg s of wrong type string
```

Не линтер, но для базовых проверок норм.

13. `go clean`

```
go clean
```

Чистит файлы, сборки и кэши.

```
go clean -cache # чистить закешированные build cache: бинари
go clean -testcache # чистить закешированные test cache
go clean -modcache # чистить закешированные модульные cache: GOMODCACHE
```

14. `go list`

```
go list
go list ./... # список всех пактов модуля
go list -m all # показать зависимости
go list -m # показать текущий модуль
go list -f '{{.Dir}}' . # показать путь к директории пакета
```

15. `go doc`

```
go doc fmt.Println
```

Показывает документацию прямо в терминале.

```
❯ go doc fmt.Printf
package fmt // import "fmt"

func Printf(format string, a ...any) (n int, err error)
    Printf formats according to a format specifier and writes to standard
    output. It returns the number of bytes written and any write error
    encountered.
```

16. `go version`

```
go version
```

Показывает версию Go

17. `go work`

```
go work init ./service-a ./service-b # создать workspace

go work use ./service-c # добавить модуль

go work sync # синхронизировать
```

Используется для workspace из нескольких модулей, когда хочется разрабатывать их вместе локально.

18. `go generate`

```
go generate ./... # пройтись и везде поискать че погенерить и сгенерить
```

Запускает генераторы кода, указанные в комментариях.

Например, в `.go` файле:

```
//go:generate mockgen -source=service.go -destination=mock_service.go
```

В результате выполнения команды будет создан файл `mock_service.go` .

Можно еще писать gen файлы, например, с помощью html/template, text/template и выполнять кодогенерацию `через go generate`

19. `go tool`

```
go tool имя_инструмента
```

Запускает внутренние инструменты Go.

```
go test -coverprofile=coverage.out ./...
go tool cover -html=coverage.out
```

20. ШПОРА:

```
go run .              — собрать во временный файл и сразу запустить
go build              — собрать бинарник
go build -o app       — собрать бинарник с именем app
go install pkg@ver    — установить бинарник в GOBIN/GOPATH/bin
go test ./...         — запустить все тесты
go test -v ./...      — тесты с подробным выводом
go test -race ./...   — тесты с race detector
go mod init <path>    — создать go.mod
go mod tidy           — привести зависимости в порядок
go get module@version — изменить зависимость в go.mod
go mod vendor         — положить зависимости в vendor/
go fmt ./...          — форматировать код
gofmt -w .            — форматировать файлы
go vet ./...          — найти вероятные ошибки
go clean -cache       — очистить build cache
go clean -modcache    — очистить кэш модулей
go list ./...         — список пакетов
go env                — настройки Go
go version            — версия Go
go doc <symbol>       — документация
go generate ./...     — запустить генераторы
go work ...           — работа с multi-module workspace
```

Типы данных

Основные встроенные типы:

- bool
- string
- int int8 int16 int32 int64
- uint uint8 uint16 uint32 uint64
- float32 float64
- complex64 complex128
- byte // alias uint8
- rune // alias int32

Функции

Функции в Go принимают 0 и более аргументов и возвращают любое количество значений.

Аргументы функций одного типа можно перечислить и указать тип за ними

```
func Sum(a int, b int) int {
  return a + b
}

func Swap(a, b string) (string, string) {
  return b, a
}
```

Возвращаемые значения могут быть именованными, тогда внутри функции их можно использовать, они будут иметь объявленный в сигнатуре тип:

```
func split(sum int) (x, y int) {
  x = sum * 4 / 9 // было 0, стало sum * 4 / 9
  y = sum - x // было 0, стало sum - x
  return
}

func main() {
  r, m := split(10)
  fmt.Println(r, m)
}
```

Переменные

Для объявления переменной используется ключевое слово var.

Переменные инициализируются значениями по умолчанию.

Для:

- numeric: 0
- bool: false
- string: ""
- slice: []
- map: nil
- struct: структура с полями нулевыми значениями
- chan: nil

```
package main

import "fmt"

var (
  y, z int // объявление переменных на уровне пакета
  i, j int = 1, 2 // объявление и инициализация на уровне пакета
)

func main() {
  var x int // объявление на уровне функции
  x = 10 // присваивание
  fmt.Println(x)

  var a, b, c bool // объявление трех переменных
  fmt.Println(a, b, c) // false, false, false

  var s string = "Hello" // объявление и присваивание
  fmt.Println(s) // Hello

  fmt.Println(y, z) // 0 0
  fmt.Println(i, j) // 1 2
}
```

Объявление и присваивание можно делать сокращенным:

```
func main() {
  var x, y, z = 1, 2, 3
  a, b, c := 1, false, "Hello"
}
```

При использовании := тип вычисляется автоматически из правой части выражения. Если сделать так:

```
func main() {
  var i int = 1
  j := i
}
```

То тип j будет таким же, как и i.

Если же справа будет число с определенной точностью, то тип будет вычисляться опираясь на точность:

```
i := 42           // int
f := 3.142        // float64
```

```
g := 0.867 + 0.5i // complex128
```

Константы

Константы можно объявлять с помощью ключевого слова const:

```
package main

import "fmt"

const (
  Pi = 3.14159265358979323846
)

const a = 10

func main() {
  const World = "Hello" // константа с типом string
  fmt.Println(Pi, World, a)
}
```

Константами могут быть только базовые типы, известные на этапе компиляции:

- numeric
- string
- bool

Структуры, мапы, функции, каналы, поинтеры, слайсы не могут быть константами.

Есть типизированные константы и не типизированные.

Untyped константы могут принимать тип, к которому их можно привести (в который она помещается) при присваивании в определенную переменную:

```
const a = 10

func main() {
  var b int = a // норм
  var c float64 = a // норм
  var d int64 = a // норм
}
```

Тут компилятор смотрит по контексту var c float64 = a -> значит x можно представить как float64.

Но если константа не помещается, то будет ошибка:

```
const a = 300

func main() {
  var b int = a // норм
  var c byte = a // ошибка, потому что byte [0, 255]
}
```

Типизированные константы могут принимать только тип, с которым они объявлены:

```
const a int = 10

func main() {
  var b int = a // норм
  var c float64 = a // ошибка
}
```

Константы нельзя объявить через :=.

Приведение типов

В Go почти нет явного приведения типов.

Почти всегда нужно писать:

```
T(value)
```

Например:

```
var a int = 10
var b int64 = int64(a)
var c float64 = float64(a)

var d int64 = a // error
```

1. Приведение числовых типов:

Числа можно приводить между собой, но может быть потеря данных:

```
var a int = 300
var b uint8 = uint8(a) // 44, тк uint8 [0, 255] -> 44 = 300 (mod 256)
```

```
var f float64 = 3.99
var a int = int(f) // 3, тк дробная часть просто отбрасывается при приведении float к int
```

2. Приведение пользовательских типов:

Допустим есть два пользовательских типа, которые под капотом могут быть основаны на одном и том же типе. Тогда все равно нужно приводить их через T(value).

```
type ID int32
type IDD int32

var id ID = 10
var idd IDD = IDD(id) // 10
var idd2 IDD = id // ошибка

ids := []ID{10, 20, 30}

//nums := []int(ids) // ошибка

// Чтобы привести слайс, нужно приводить каждый элемент

nums := make([]int, len(ids))
for i, id := range ids {
  nums[i] = int(id)
}
```

Можно не создавать новый тип, а создать alias для уже имеющегося типа:

```
type ID = int32 // буквально название для int32, а не новый тип

var a int32 = 10
var id ID = a // норм
```

Если свой тип - структура, то их можно привести только если у них одинаковые названия, типы и теги полей:

```
type User struct {
  id int32
  name string `json:"name"`
}
```

```
type Person struct {
  id int32
  name string `json:"name"`
}

type User2 struct {
  id int32
  fullname string `json:"fullname"`
  age int32
}

var u User = User{id: 10, name: "Vlad"}
p := Person(u) // норм
u2 := User2(u) // ошибка
```

3. Type assertion (утверждение типа):

Механзим динамической проверки типа, когда мы говорим "Я утверждаю, что внутри x лежит type"

```
value, ok := something.(Type)
```

```
var x interface{} = 10
var y int = x.(int) // 10
var z string = x.(string) // panic
```

При этом если этот тип там не лежит, то будет паника

Для этого есть механизм безопасной проверки типа:

```
var x int = 10
val, ok := x.(string)
if !ok {
  fmt.Println("x is not int")
}
```

То есть type assertion утверждает, что значение интерфейса x содержит конкретный тип T и если да, то помещает базовое значение x типа T в переменную val.

Если же мы небезопасно проверяем и в x лежит другой тип, то будет паника.

Если же мы безопасно проверяем, то вернется ok=false, а в val будет zero-value типа T.

```
val, ok := x.(T)
```

type switches

Для проверки типа есть конструкция со switch:

```
switch v := i.(type) {
  //
}
```

Без switch v:=i.(type) не работает.

```
package main

import "fmt"

func do(i interface{}) {
    switch v := i.(type) {
    case int:
        fmt.Printf("Twice %v is %v\n", v, v*2)
    case string:
        fmt.Printf("%q is %v bytes long\n", v, len(v))
    default:
        fmt.Printf("I don't know about type %T!\n", v)
    }
}

func main() {
    do(21)
    do("hello")
    do(true)
}
```

4. Сравнение двух значений:

Чтобы сравнить два значения, нужно понять:

- является ли тип сравнимым
- можно ли сравнить эти два значения друг с другом

Сравниваемые через == и !=:

- int
- float64

- string
- bool
- pointers
- channels
- interfaces
- structs, если все поля сравнимы
- arrays, если все элементы сравнимы

Не сравниваемые через == и !=:

- maps
- slices
- functions

Но можно их сравнить с nil:

```
var m map[string]int
fmt.Println(m == nil) // true

var s []int
fmt.Println(s == nil) // true

var c []int = []int{}
fmt.Println(c == nil) // false
```

Структуры нельзя сравнить, если поле структуры не сравнимо:

```
type User struct {
  id int32
  names []string
}

type Us struct {
  Vals [3]int
}

u1 := User{id: 10, name: []string{"Vlad", "Vlados"}}
u2 := User{id: 10, name: []string{"Vlad"}}
fmt.Println(u1 == u2) // ошибка


us1 := Us{Vals: [3]int{1, 2, 3}}
us2 := Us{Vals: [3]int{1, 2, 3}}
fmt.Println(us1 == us2) // true
```

Перед сравнением нужно приводить типы:

```
var a int = 10
var b int64 = 10

fmt.Println(a == b) // ошибка
fmt.Println(int64(a) == b) // true
```

Циклы

В Go есть только for циклы.

В нем есть три основных компонента:

- инициализация утверждения/переменной: исполняется перед первой итерацией
- условие цикла: проверяется пере каждой итерацией
- пост утверждение: выполняется после каждой итерации

Простейший цикл:

```
for i := 0; i < 10; i++ {
  fmt.Println(i)
}
```

Все три компонента опциональны, тогда будет бесконечный цикл:

```
for {
  fmt.Println("Hello")
}
```

Цикл while можно организовать так:

```
var i int = 0
for i < 10 {
  fmt.Println(i)
  i++
}

// или
i = 0
for ; i < 10; {
  fmt.Println(i)
  i++
}
```

Менять счетчик цикла в цикле можно, но нужно быть внимательным:

```
func main() {
    i:=0

    for ;i < 10; i++{
        fmt.Println(i)
        i++
    }

  // 0 2 4 6 8
}
```

Условные конструкции

if-else

Синтаксис:

```
if value < 10{
  return true
}
```

или с предварительным вычислением

```
if val := SomeFunc(); val < 10{
  return true
} else {
  fmt.Printf("val is not less than 10: %d", val) // вычисленное значение доступно в блоке else
  return false
}

fmt.Println(val) // ошибка, потому что val тут уже не доступен
```

Чтобы val был доступен за пределами if-else конструкции, нужно его отдельно найти:

```
val := SomeFunc()
if val < 10{
  return true
} else {
  fmt.Printf("val is not less than 10: %d", val)
  return false
}
```

```
fmt.Println(val) // норм
```

Практический пример

```
if resp, err := GetUser(ctx, req); err != nil{
  return nil, fmt.Errorf("error getting user: %w", err)
}
```

switch - case

Синтаксис:

```
switch init; expression {
  case value1:
    // code
  case value2:
    // code
  default:
    // code
}
```

Пример:

```
package main

import (
    "fmt"
    "runtime"
)

func main() {
    fmt.Print("Go runs on ")
    switch os := runtime.GOOS; os {
    case "darwin":
        fmt.Println("macOS.")
    case "linux":
        fmt.Println("Linux.")
    default:
        // freebsd, openbsd,
        // plan9, windows...
        fmt.Printf("%s.\n", os)
    }
}
```

Или такой пример:

```
resp, err := s.db.GetUserType(ctx, req)
if err != nil {
  return err
}

switch resp.UserType {
  case constants.UserTypeAdmin:
    return fmt.Sprintf("Users %s type is admin", req.Username)
  case constants.UserTypeUser:
    return fmt.Sprintf("Users %s type is user", req.Username)
  default:
    return fmt.Sprintf("Users %s type is unknown", req.Username)
}
```

Также switch можно использовать без условия, тогда внутренние проверки case будут выполняться в любом случае, так как switch {} <=> switch true {}:

```
package main

import (
    "fmt"
    "time"
)

func main() {
    t := time.Now()
    switch {
    case t.Hour() < 12:
        fmt.Println("Good morning!")
    case t.Hour() < 17:
        fmt.Println("Good afternoon.")
    default:
        fmt.Println("Good evening.")
    }
}
```

Defer

Defer позволяет выполнить какую-то функцию после завершения текущей функции (даже если она завершится с panic).

Множественные defer вызовы складываются в стек и выполняются после завершения в порядке LIFO (last-in-first-out).

Defer привязан к функции, а не к блоку кода и выполнится только после завершения функции.

```
func main() {
    defer fmt.Println("world!")
    fmt.Println("Hello")
}
// Hello world!
```

В примере выше выполняется defer после завершения функции main.

```
func main() {
  fmt.Println("Counting")

  for i := 0; i < 10; i+=2 {
    defer fmt.Println(i)
  }

  fmt.Println("Done")
}
// Counting
// Done
// 8 6 4 2 0
```

Правило:

Аргументы deferred-функции вычисляются сразу в момент выполнения строки defer.

Пример:

```
func main() {
  i := 0

  defer fmt.Println(i) // сюда положится i=0

  for ; i < 10; i++{
    //code
  }

  defer fmt.Println(i) // сюда положится i=10

  fmt.Println("RESULT:")
}
// RESULT:
// 10
// 0
```

Практичный пример:

```
req, err := json.NewDecoder(r.Body).Decode(&req)
if err != nil {
  return nil, fmt.Errorf("error decoding request: %w", err)
}
defer r.Body.Close()
//code

// Как только завершили работу с телом запроса, закрываем поток
```

Замыкания

Замыкание - это функция, которая "запомнила" переменные из внешней области видимости и может их использовать позже, даже если внешняя функция завершилась.

При этом замыкание запоминает переменную, а не ее значение. То есть функция будет выполняться с тем значением переменной, которое лежит на момент ее вызова, а не создания.

Пример:

```
func main() {
  i := 0

  f := func() {
    fmt.Println(i)
  }

  i = 10
  f()
  // 10, т.к. в момент вызова f i=10
}
```

Если же с defer:

```
func main() {
  i := 0

  defer func() {
    fmt.Println(i)
  }()

  i = 10
}
// 10, т.к. это замыкание (используется внешняя переменная) и в момент вызова defer i=10
```

Без замыкания:

```
func main() {
  i := 0

  defer
    fmt.Println(i)

  i = 10
}
// 0, т.к. это не замыкание и в момент укладывания defer в стек i=0
```

Типичный пример замыкания (Счетчик):

```
func NewCounter() func() int {
    count := 0

    return func() int {
        count++
        return count
    }
}

func main() {
    c1 := NewCounter()
    c2 := NewCounter()

    fmt.Println(c1()) // 1
    fmt.Println(c1()) // 2
    fmt.Println(c1()) // 3

    fmt.Println(c2()) // 1
    fmt.Println(c2()) // 2
}
```

Указатели

Указатель хранит адрес памяти, в которой хранится значение переменной.

```
var num int = 10
var ptr *int = &num// указатель на адрес памяти, в которой хранится значение переменной num ти
```

Zero value указателя nil:

```
var ptr *int
```

```
fmt.Println(ptr) // <nil>
```

Операция генерации указателя на переменную:

```
i := 42
p := &i
fmt.Println(p) // 0x104f0c0
```

Операция разыменовывания указателя:

```
i := 42
p := &i
*p = 10
fmt.Println(i) // 10
```

Также можно создать указатель через new(T), возвращающее *T для структур, массивов, словарей, встроенных типов и всего остального:

```
// создадим указатель на int
i := new(int)
fmt.Println(i) // 0x104f0c0

// положим в память, на которую он указывает значение 10
*i = 10
fmt.Println(*i) // 10

// создадим переменную a = значение по указателю i
a := *i
fmt.Println(a) // 10

// возьмем адрес у a, он будет отличаться от i, потому что это новая переменная с другим адрес
aptr := &a
fmt.Println(aptr) // 0x10000c0
```

В Go нет адресной арифметики, как в C.

Указатели нужны для изменения значения переменной внутри функции и для избежания копирования.

Структуры

Структура - составной тип данных, содержащий набор полей.

Структуры объявляются так:

```
type Name struct {
  field1 type
  field2 type
  ...
}
```

Структуры могут быть анонимными:

```
func main() {
  example := struct{
    Name string
    Age int
  } { // структура с полями Name и Age
    Name: "Vlad",
    Age: 30,
  }

  fmt.Println(example.Name) // Vlad
  fmt.Println(example.Age) // 30
}
```

Обращение к полям структуры осуществляется через точку.

При объявлении полей структуры можно не именовать первые поля типов, но если полей одного типа много, то именование обязательно:

```
type User struct {
  string // норм
  company string
  int
}
```

```
type User struct {
  string
  string //ошибка redeclared
  Age int
}
```

Обращаться к полям структуры можно через указатель:

```
type User struct {
  Name string
  Age int
}
```

```
func main() {
  user := &User{
    Name: "Vlad",
    Age: 30,
  }

  fmt.Println(user.Name) // Vlad
  fmt.Println(user.Age) // 30

  fmt.Println((*user).Name) // Vlad
  fmt.Println((*user).Age) // 30
}
```

Также можно обращаться к полям структуры через указатель:

```
type User struct {
  Name string
  Age int
}

func main() {
  user := &User{
    Name: "Vlad",
    Age: 30,
  }

  var u_age *int = &user.Age
  fmt.Println(*u_age) // 30

  *u_age = 40
  fmt.Println(user.Age) // 40
}
```

Структуру можно инициализировать через new() или напрямую:

```
func main() {
  user1 := User{
    Name: "Vlad",
    Age: 30,
  }

  user2 := new(User) // вернет указатель на User

  fmt.Println(user2) // {"", 0}

  user2.Name = "Vlad"
  user2.Age = 30

  fmt.Println(user1.Name) // Vlad
  fmt.Println(user1.Age) // 30
```

```
  fmt.Println(user2.Name) // Vlad
  fmt.Println(user2.Age) // 30
}
```

При инициализации будет структура с полями с zero-value.

Методы структур

В Go нет классов.

Метод - функция, привязанная к типу (структуре).

В Go метод - функция со специальным аргументом reciever (приемник).

Сигнатура метода:

```
func (приемник) FuncName(параметры) результат {
  // код
}
```

Приемник представляет собой имя структуры, в которой определен метод.

Можно в качестве приемника передать указатель на структуру или саму структуру.

При передаче указателя в качестве приемника можно будет внутри метода изменять поля структуры.

При передаче структуры произойдет копирование и изменения в копии не повлияют на оригинал.

```
package main

import "fmt"

type User struct {
    Age  int
    Name string
}

func (u *User) UpdateAgePtr() {
    u.Age++
}

func (u User) UpdateAge() {
    u.Age++
}
```

```
func main() {
    u := new(User)

    u.Age = 20
    u.Name = "Vlad"

    u.UpdateAge()
    fmt.Println(u) //&{20 Vlad} т.к. апдейтнулась копия

    u.UpdateAgePtr()
    fmt.Println(u) //&{21 Vlad} т.к. апдейтнулась структура

    u2 := User{}
    u2.UpdateAge()
    fmt.Println(u2) // {0 } т.к. апдейтнулась копия

    u2.UpdateAgePtr()
    fmt.Println(u2) // {1 } т.к. апдейтнулась структура
}
```

Полями пользователя могут быть и другие структуры:

```
type User struct {
  Name string
  Age int
  Address struct {
    Street string
    City string
  }
}
```

Методы можно писать не только для структур, но и для любого типа.

```
type MyError error

func (e MyError) Error() string {
  return fmt.Errorf("my error: %s", e.Error())
}
```

Все методы для типа должны быть определены в том же пакете, где и сам метод.

Если метод принимает указатель (в качестве приемника),

то мы можем взять указатель от структуры и вызвать метод от него напрямую,

либо просто вызвать метод от структуры, тогда Go интерпретирует его как указатель на структуру:

```
func (u *User) UpdateAgePtr() {
  u.Age++
}

func main() {
  u := User{
    Name: "Vlad",
    Age: 30,
  }

  u.UpdateAgePtr() // представится как (&u).UpdateAgePtr()

  fmt.Println(u) // {31 Vlad}

  p := &u
  p.UpdateAgePtr() // представится как (&u).UpdateAgePtr()

  fmt.Println(u) // {32 Vlad}
}
```

Если же метод принимает структуру как приемник, и мы вызовем его от указателя на структуру, то Go его разыменует:

```
func (u User) PrintAge() {
  fmt.Println(u.Age)
}

func main() {
  u := User{
    Name: "Vlad",
    Age: 30,
  }

  u.PrintAge() // 30

  p := &u
  p.PrintAge() // 30, представится как (*p).PrintAge()
}
```

Как выбрать ptr или по значению?

ptr: если нам нужно избежать копирования больших структур и нужно менять поля структуры

value: если структура не большая и мы ее не меняем

Лушче использовать что-то одно, потому что когда дело доходит до интерфейсов, могут возникнуть проблемы, что

для каких-то типов приемник *T, а для других T:

```
package main

import (
    "fmt"
    "math"
)

type Abser interface {
    Abs() float64
}

func main() {
    var a Abser
    f := MyFloat(-math.Sqrt2)
    v := Vertex{3, 4}

    a = f  // a MyFloat implements Abser
    a = &v // a *Vertex implements Abser

    // In the following line, v is a Vertex (not *Vertex)
    // and does NOT implement Abser.
    a = v // Тут будет ошибка, так как *Vertex реализует интерфейс, а Vertex нет

    fmt.Println(a.Abs())
}

type MyFloat float64

func (f MyFloat) Abs() float64 {
    if f < 0 {
        return float64(-f)
    }
    return float64(f)
}

type Vertex struct {
    X, Y float64
}

func (v *Vertex) Abs() float64 {
    return math.Sqrt(v.X*v.X + v.Y*v.Y)
}
```

Массивы и слайсы

Массив - это структура данных, содержащая набор элементов одного типа, лежащих подряд в памяти.

Массивы в Go не могут менять размер.

Слайс - оболочка над массивом: динамический массив, можно воспринимать как указатель на массив.

Zero-value: nil

```
func main() {
    var s []int // s == nil
    fmt.Println(len(s), cap(s)) // 0 0

    sl := make([]int, 0) // s != nil
    fmt.Println(len(sl), cap(sl)) // 0 0

    sl2 := []int{} // s != nil
    fmt.Println(len(sl2), cap(sl2)) // 0 0
}
```

```
func main() {
    primes := [6]int{2, 3, 5, 7, 11, 13} // массив из 6 элементов

    var s []int = primes[1:4] // срез с первого по третий элемент
    fmt.Println(s)

  // Если поменять элемент среза, то поменяется элемент базового массива
  s[0] = 10
  fmt.Println(primes) // [2 10 5 7 11 13]
}
```

Добавление элементов в слайс

Добавление элементов в слайс выполняется через метод append:

```
func main() {
    primes := [6]int{2, 3, 5, 7, 11, 13} // массив из 6 элементов

    s := primes[1:4] // срез с первого по третий элемент
    fmt.Println(s)

    s = append(s, 17) // добавляем элемент в конец среза
    fmt.Println(s)

    s = append(s, 19) // добавляем элемент в конец среза
```

```
    fmt.Println(s)
}
```

Даже если слайс был nil, append под капотом его проинициализирует и добавит элемент.

Удаление элементов из слайса

Удаление элементов из слайса можно сделать через append+срез:

```
users := []string{"Bob", "Alice", "Kate", "Sam", "Tom", "Paul", "Mike", "Robert"}
//удаляем 4-й элемент
var n = 3
users = append(users[:n], users[n+1:]...)
fmt.Println(users)      //["Bob", "Alice", "Kate", "Tom", "Paul", "Mike", "Robert"]
```

Копирование слайса

Копирование слайса можно сделать через

```
func copy( destination, source [] T) int
```

copy копирует минимум длин source, destination. То есть если длина source = 3, длина destination = 5, то копируется только первые 3 элементов destination в source. Если же наоборот, то скопируются три элемента из destination в source, остальные элементы source будут zero-value.

Сортировка слайса

Для сортировки срезов используют пакет sort.

В нем есть функции для сортировки, поиска элементов и т.д.

```
package main

import (
    "fmt"
    "sort"
)

func main() {
    s := []int{2, 3, 5, 7, 11, 13}
  s1 := []string{"Vlad", "Alex", "Ivan"}
```

```
    sort.Ints(s)
    fmt.Println(s)

    sort.Strings(s1)
    fmt.Println(s1)

  fmt.Println(sort.SearchInts(s, 3)) // 1 - индекс элемента в слайсе

  intSlice := [] int {11, 22, 33, 44, 55, 66}
  sort.Sort(sort.Reverse(sort.IntSlice(intSlice)))
  fmt.Println(intSlice)  // [66 55 44 33 22 11]
}
```

Сравнение слайсов, пакет reflect

Для сравнения слайсов используется пакет reflect.

Слайсы равны, если они хранят данные одного типа и их элементы равны.

```
package main
import (
    "fmt"
    "reflect"   // для сравнения срезов
)

func main() {

    slice1 := [] int{1, 2, 3, 4}
    slice2 := []string{"Tom", "Bob", "Sam"}
    slice3 := [] int{1, 2, 3}
    slice4 := [] int{1, 2, 3, 4}

    fmt.Println("slice1 == slice2:", reflect.DeepEqual(slice1, slice2))     // false
    fmt.Println("slice1 == slice3:", reflect.DeepEqual(slice1, slice3))     // false
    fmt.Println("slice1 == slice4:", reflect. DeepEqual(slice1, slice4))    // true
}
```

Инициализация слайса

Инициализация слайса через метод make:

```
имя_среза := make([] тип_элементов_среза, длина_среза, емкость_среза)
```

make аллоцирует zero-valued массив и возвращает слайс на него.

Метод len возвращает количество элементов в слайсе;

Метод cap возвращает количество места, которое выделено для слайса.

Если при append len становится > cap, то происходит реалокация массива и все элементы копируются в новый массив.

Слайс можно объявить как массив без указания количества элементов:

```
func main() {
    primes := make([]int, 6, 10) // слайс из 6 элементов

    fmt.Println(len(primes)) // 6
  fmt.Println(cap(primes)) // 10
  fmt.Println(primes) // [0 0 0 0 0 0] , т.к. длина 6

  nums := make([]int, 0, 20) // пустой слайс, длина 0, капас 20
  fmt.Println(nums) // [], т.к. длина 0

  ns := []int{} // пустой слайс
}
```

```
package main

import "fmt"

func main() {
    s := []int{2, 3, 5, 7, 11, 13}

    s = s[1:4]
    fmt.Println(s)

    s = s[:2]
    fmt.Println(s)

    s = s[1:]
    fmt.Println(s)
}
//[3 5 7]
//[3 5]
//[5]
```

!!! ОЧЕНЬ ВАЖНО !!!

Слайс под капотом имеет примерно такую структуру:

```
struct {
  length int
  capacity int
  ptr *int
}
```

len - длина среза: сколько элементов в слайсе

capacity - размер среза: сколько элементов может быть в слайсе до реалокации

ptr - указатель на первый элемент среза

При этом итерироваться можно только в пределах len массива, если выйти за len -> panic

ОЧЕНЬ ВАЖНЫЙ ПРИМЕР:

```
package main

import "fmt"

func main() {
  // Тут у нас выделилось место для слайса из 6 элементов
  // s указывает на первый элемент среза
    s := []int{2, 3, 5, 7, 11, 13}
    printSlice(s) // len=6 cap=6 [2 3 5 7 11 13]

    // Откинем все элементы среза и сделаем его пустым
  // тут s по прежнему указывает на первый элемент среза
    s = s[:0]
    printSlice(s) // len=0 cap=6 []

    // Увеличим длину среза до 4
  // Так как у нас элементы лежат подряд, то мы теперь можем
  // опять увидеть их до 4го
    s = s[:4]
    printSlice(s) // len=4 cap=6 [2 3 5 7]

    // Дропнем первые 2 элемента среза
  // Теперь s сдвинулся на второй элемент среза
  // len 2, так как до этого был 4 и мы сдвинулись на 2
  // cap 4, так как изначально был 6, но теперь срез со второго элемента
    s = s[2:]
    printSlice(s) // len=2 cap=4 [5 7]
}

func printSlice(s []int) {
    fmt.Printf("len=%d cap=%d %v\n", len(s), cap(s), s)
}
```

Матрицы - слайсы слайсов || массивы массивов

```
package main

import (
    "fmt"
    "strings"
)

func main() {
    // Create a tic-tac-toe board.
    board := [][]string{
        []string{"_", "_", "_"},
        []string{"_", "_", "_"},
        []string{"_", "_", "_"},
    }

    // The players take turns.
    board[0][0] = "X"
    board[2][2] = "O"
    board[1][2] = "X"
    board[1][0] = "O"
    board[0][2] = "X"

    for i := 0; i < len(board); i++ {
        fmt.Printf("%s\n", strings.Join(board[i], " "))
    }
}
```

Итерирование

Для итерирования в цикле или по мапе (дальше и для каналов) используется range. Результатом выполнения будет два значения при итерации:

- первое: индекс/ключ
- второе: копия значения

```
package main

import "fmt"

func main() {
    var s = []int{2, 3, 5, 7, 11, 13}

    for i, v := range s {
        fmt.Printf("i=%d, v=%d\n", i, v)
    }
```

```
    // i=0, v=2
    // i=1, v=3
    // i=2, v=5
    // i=3, v=7
    // i=4, v=11
    // i=5, v=13

  var m = map[string]int{"Vlad": 10, "Alex": 20, "Ivan": 30}

  for k, v := range m {
    fmt.Printf("k=%s, v=%d\n", k, v)
  }

  // k=Vlad, v=10
  // k=Alex, v=20
  // k=Ivan, v=30
}
```

В Go можно пропускать индекс или значение если не нужно:

```
s := []int{2, 3, 5, 7, 11, 13}

for _, v := range s {
  fmt.Println(v) // 2, 3, 5, 7, 11, 13
}

for i := range s {
  fmt.Println(i) // 0, 1, 2, 3, 4, 5
}

for i, _ := range s {
  fmt.Println(i) // 0, 1, 2, 3, 4, 5
}
```

Map

Мапа мапит ключи и значения.

Zero-value: nil

Ключ могут быть только сравнимыми типами (которые можно сравнить через == или !=).

Значения могут быть любыми типами.

Подробнее про сравнение типов можно прочитать в разделе о типах. Таким образом структура может быть ключом в мапе только если все ее поля сравнимы и т.д.

Для получения размера мапы используется функция len, возвращающая количество пар ключ-значение в мапе.

Брать адрес элемента мапы нельзя, потому что под капотом может быть перераспределение бакетов, перемещение данных и тд.

Из nil map можно читать (вернется zero-value значения), но в нее нельзя писать (паника):

```
var m map[string]int
fmt.Println(m["Vlad"]) // 0

m["Vlad"] = 10 // panic
```

Для инициализации мапы используется ключевое слово make или инициализатор/литерал:

```
m1 := make(map[string]int)

m2 := map[string]int{}
```

Вставка и обновление

```
m := make(map[string]int)
m["Vlad"] = 10
fmt.Println(m) // {"Vlad": 10}

m["Vlad"] = 20
fmt.Println(m) // {"Vlad": 20}
```

Пример со структурой:

```
type User struct {
  Name string
  Age int
}

m := map[string]User{
  "Vlad": User{Name: "Vlad", Age: 30},
}

// Допустим мы хотим обновить в мапе значение поля Age для ключа Vlad
// При получени значения из мапы по ключу мы получаем КОПИЮ
// То есть нужно поменять копию и обновить значение в мапе
userForUpdate := m["Vlad"]

userForUpdate.Age = 40
```

```
m["Vlad"] = userForUpdate

// Либо можно хранить указатель на структуру
// Тогда можно изменить по указателю

mm := map[string]*User{
  "Vlad": &User{Name: "Vlad", Age: 30},
}

userForUpdatePtr = mm["Vlad"]
userForUpdatePtr.Age = 40

fmt.Println(mm["Vlad"]) // &{40 Vlad}
```

Удаление

Удаление ключа из мапы происходит через delete функцию. Ее можно вызвать даже для nil мапы.

```
m := make(map[string]int)
m["Vlad"] = 10
delete(m, "Vlad")
fmt.Println(m) // {}
```

Чтобы отчистить мапу, есть функция clear:

```
m := make(map[string]int)
m["Vlad"] = 10
clear(m)
fmt.Println(m) // map[]
fmt.Println(len(m)) // 0
```

Получение значения по ключу

При получении значения по ключу можно получить флаг ok, который указывает на то, что ключ существует в мапе.

Если ключа нет, то вернется zero-value значения, флаг будет false.

```
m := make(map[string]int)
m["Vlad"] = 10

elem, ok := m["Vlad"] // elem=10, ok=true
```

```
elem, ok = m["Alex"] // elem=0, ok=false
```

Перебор мапы

При переборе мапы порядок не гарантируется.

Для перебора мапы используется range функция:

```
m := map[string]int{
  "Vlad": 10,
  "Alex": 20,
  "Ivan": 30,
}

for key, value := range m {
  fmt.Printf("key=%s, value=%d\n", key, value)
}

// key=Vlad, value=10
// key=Alex, value=20
// key=Ivan, value=30
```

Важно!!!

Мапа сама по себе ссылочный тип, но не pointer (схоже со slice).

То есть map-структура под капотом ссылается на runtime хеш-таблицу.

Пример:

Мапа передается по значению, затем мы внутри функции меняет значение по ключу и ничего не возвращаем.

```
func change(m map[string]int) { // копируется map-структура, она ссылается на исходную хеш-таб
    m["go"] = 1
}

func main() {
    data := make(map[string]int)

    change(data)
```

```
    fmt.Println(data) // map[go:1]
}
```

Но если я внутри заменю мапу, то внешняя переменная не изменится:

```
func change(m map[string]int) { // также копируется map-структура, она ссылается на исходную х
    m = make(map[string]int) // здесь теперь копия начинает ссылаться на другую хеш-таблицу (н
  m["go"] = 1 // здесь мы меняем значение в новой хеш-таблице, а не в исходной
  fmt.Println(m) // map[go:1]
}

func main() {
    data := make(map[string]int)

    change(data)

    fmt.Println(data) // map[]
}
```

Для того, чтобы внутри функции изменить саму мапу и при этом изменить исходную, нужно возвращать мапу:

```
func change(m map[string]int) map[string]int {
    m = make(map[string]int)
  m["go"] = 1

  fmt.Println(m) // map[go:1]

    return m
}

func main() {
    data := make(map[string]int)

    data = change(data) // теперь ссылается на новую хеш-таблицу

    fmt.Println(data) // map[go:1]
}
```

Мапы нельзя сравнивать друг с другом, только с nil.

Для сравнения содержимого мап можно использовать пакет maps:

```
package main

import (
    "fmt"
```

```
    "maps"
)

func main() {
    a := map[string]int{"go": 1}
    b := map[string]int{"go": 1}

    fmt.Println(maps.Equal(a, b)) // true
}
```

Concurrency

Мапы небезопасны для записи, нужно использовать RWMutex, Mutex.

Интерфейсы

Интерфейс в Go - тип, который определен как множество сигнатур методов.

В переменную типа interface T можно хранить значение любого типа, который реализует интерфейс T.

Пример:

```
package main

import (
    "fmt"
    "math"
)

type Abser interface {
    Abs() float64
}

func main() {
    var a Abser
    f := MyFloat(-math.Sqrt2)
    v := Vertex{3, 4}

    a = f  // a MyFloat implements Abser
    a = &v // a *Vertex implements Abser

    // In the following line, v is a Vertex (not *Vertex)
    // and does NOT implement Abser.
    a = v
```

```
    fmt.Println(a.Abs())
}

type MyFloat float64

func (f MyFloat) Abs() float64 {
    if f < 0 {
        return float64(-f)
    }
    return float64(f)
}

type Vertex struct {
    X, Y float64
}

func (v *Vertex) Abs() float64 {
    return math.Sqrt(v.X*v.X + v.Y*v.Y)
}
```

Тип реализует интерфейс автоматически, если он реализует методы интерфейса.

В Go не пишут явно type MyFloat float64 implements Abser и т.д.

```
package main

import "fmt"

type I interface {
    M()
}

type T struct {
    S string
}

// This method means type T implements the interface I,
// but we don't need to explicitly declare that it does so.
func (t T) M() {
    fmt.Println(t.S)
}

func main() {
    var i I = T{"hello"}
    i.M()
}
```

В общем, значение интерфейса можно воспринимать как кортеж их значения и конкретного типа.

Таким образом в переменной типа interface содержится значение конкретного типа, который его реализует.

При этом вызов метода интерфейса влечет вызов одноименного метода конкретно этого типа.

```
package main

import (
    "fmt"
    "math"
)

type I interface {
    M()
}

type T struct {
    S string
}

func (t *T) M() {
    fmt.Println(t.S)
}

type F float64

func (f F) M() {
    fmt.Println(f)
}

func main() {
    var i I

    i = &T{"Hello"}
    describe(i)
    i.M()

    i = F(math.Pi)
    describe(i)
    i.M()
}

func describe(i I) {
    fmt.Printf("(%v, %T)\n", i, i)
}

// (&{Hello}, *main.T)
// Hello
// (3.141592653589793, main.F)
// 3.141592653589793
```

Если конкретное значение интерфейса nil, то метод вызовется с nil reciever.

Если сам интерфейс nil, то вызов метода падает с ошибкой, так как он не понимает, какой метод вызывать.

```
package main

import "fmt"

type I interface {
    M()
}

func main() {
    var i I
    describe(i)
    i.M()
}

func describe(i I) {
    fmt.Printf("(%v, %T)\n", i, i)
}
```

Empty interface

Интерфейс без единого метода interface{} - empty interface.

В него можно положить значение любого типа, потому что любой тип реализует как минимум 0 методов.

Для примера, fmt.Print принимает любое количество аргументов типа interface{}.

```
package main

import "fmt"

func main() {
    var i interface{}
    describe(i)

    i = 42
    describe(i)

    i = "hello"
    describe(i)
}

func describe(i interface{}) {
```

```
    fmt.Printf("(%v, %T)\n", i, i)
}

// (<nil>, <nil>)
// (42, int)
// (hello, string)
```

Stringer

Одним из основных интерфейсов является Stringer, который объявлен в пакете fmt.

Stringer позволяет привести значение типа T в строку с помощью метода String() string.

Он используется в fmt для представления значений типа T в строку.

```
type Stringer interface {
    String() string
}
```

Пример:

```
package main

import "fmt"

type Person struct {
    Name string
    Age  int
}

func (p Person) String() string {
    return fmt.Sprintf("%v (%v years)", p.Name, p.Age)
}

func main() {
    a := Person{"Arthur Dent", 42}
    z := Person{"Zaphod Beeblebrox", 9001}
    fmt.Println(a, z)
}

// Arthur Dent (42 years) Zaphod Beeblebrox (9001 years
```

Еще про интерфейсы

- Интерфесы могут быть как constraint для generic типов (подробнее в разделе про Generics)

- Интерфейсы можно встраивать друг в друга, тогда тип T реализует интерфейс I1, в который встроили I2, если реализует оба интерфейса.
- Интерфейсы могут иметь методы, другие интерфейсы, типы (но тогда их можно использовать только как constraint для generic и нельзя передавать в переменных), но не может иметь поля.

Errors

В Go программы выражают ошибку выполнения с помощью типа error.

error - встроенный интерфейс, как Stringer.

```
type error interface {
    Error() string
}
```

Функции обычно должны возвращать error value, затем мы должны проверять является ли ошибка nil и если нет, то вызвать код, который ее должен обработать.

Пример:

```
package main

import "fmt"

func main() {
    var err error
    if err = SomeFunc(); err != nil {
        fmt.Println(err)
    }
}
```

Или более приближенный пример к реальности:

```
func (s *Service) GetUser(ctx context.Context, req *GetUserRequest) (*User, error) {
  resp, err := s.db.GetUserType(ctx, req)
  if err != nil {
    if errors.Is(err, sql.ErrNoRows) {
      return nil, fmt.Errorf("%w: user not found", consts.ErrNotFound)
    }

    return nil, handleDBError(err)
  }

  // ...
```

```
  return user, nil
}

//...

func handleDBError(err error) error {
  if errors.Is(err, sql.ErrNoRows) {
    return fmt.Errorf("%w: not found: %v", consts.ErrNotFound, err)
  }

  if errors.Is(err, sql.ErrTxDone) {
    return fmt.Errorf("%w: transaction already done: %v", consts.ErrInternal, err)
  }

  //...

  return fmt.Errorf("%w: %v", consts.ErrInternal, err)
}
```

Можно написать свой тип ошибки, который реализует интерфейс error и работать с ним:

```
type UnifiedError struct {
  Code    int
  Message string
  Source string
}

func (e UnifiedError) Error() string {
  return fmt.Sprintf("error: code=%d, message=%s, source=%s", e.Code, e.Message, e.Source)
}

func (s *service) GetUser(ctx context.Context, req *GetUserRequest) (*User, error) {
  resp, err := s.db.GetUserType(ctx, req)
  if err != nil {
    if errors.Is(err, sql.ErrNoRows) {
      return nil, UnifiedError{
        Code:    consts.ErrNotFound,
        Message: fmt.Sprintf("%w: user not found", consts.ErrNotFound),
        Source:  err.Error(),
      }
    }

    return nil, UnifiedError{
      Code:    consts.ErrInternal,
      Message: fmt.Sprintf("%w: unexpected error", consts.ErrInternal),
      Source:  consts.AdminServiceSourceName,
    }
  }

  // ...
```

```
  return user, nil
}
```

Reader

Интерфейс Reader представляет собой чтение до конца потока данных.

В стандартной библиотеке есть много его реализаций для файлов, сетевых подключений и др.

Для считывания данных обычно используется io.Reader, который имеет метод:

```
func Read(p []byte) (n int, err error)
```

Функция заполняет массив байт p данными из потока и возвращает количество заполненных байт и ошибку.

Если в потоке нет данных (или не осталось), то функция вернет ошибку io.EOF.

Есть реализация для считывания строк из пакета strings: strings.Reader

```
package main

import (
    "fmt"
    "io"
    "strings"
)

// В пакете strings такая реализация Reader:
// A Reader implements the [io.Reader], [io.ReaderAt], [io.ByteReader], [io.ByteScanner],
// [io.RuneReader], [io.RuneScanner], [io.Seeker], and [io.WriterTo] interfaces by reading
// from a string.
// The zero value for Reader operates like a Reader of an empty string.
// type Reader struct {
//  s        string
//  i        int64 // current reading index
//  prevRune int   // index of previous rune; or < 0
// }

func main() {
    r := strings.NewReader("Hello, Reader!") // возвращает strings.Reader

    b := make([]byte, 8)
    for {
        n, err := r.Read(b)
        fmt.Printf("n = %v err = %v b = %v\n", n, err, b)
        fmt.Printf("b[:n] = %q\n", b[:n])
```

```
        if err == io.EOF {
            break
        }
    }
}
// n = 8 err = <nil> b = [72 101 108 108 111 44 32 82]
// b[:n] = "Hello, R"
// n = 6 err = <nil> b = [101 97 100 101 114 33 32 82]
// b[:n] = "eader!"
// n = 0 err = EOF b = [101 97 100 101 114 33 32 82]
// b[:n] = ""
```

Generics и Параметры типа

Параметры типа в функциях

До появления generics в Go приходилось писать функции с одинаковой логикой под каждый тип. С появлением generics мы теперь можем использовать параметры типа:

```
func PrintInt(x int) {
  fmt.Println(x)
}

func PrintString(x string) {
  fmt.Println(x)
}

func Print[T any](x T) {
  fmt.Println(x)
}

func main() {
  PrintInt(10) // 10
  PrintString("10") // 10
  Print("10") // 10
  Print(10) // 10

  // Можно еще писать:
  // Print[int](10)
  // Print[string]("10")
  // Но даже если написать без этого, Go сам все это подставит
  // Print(10) -> Go сам понимает, что T - int
}
```

Синтаксис такой:

```
func Something[T constraint_interface1, S constraint_interface2](
  arg1 T,
  arg2 S,
  arg3 int,
  arg4 string,
  ...,
  ) (T, int, ...) {
  // code
}
```

То есть после названия функции мы в квадратных скобках указываем ограничение (constraint) типа interface для переменных типа T и других типов, которые мы хотим использовать как generics.

Из встроенных есть два constraint интерфейса:

- any - любой тип (alias для interface{})
- comparable - типы, которые можно сравнивать с помощью == и !=

Пример:

```
package main

import (
    "fmt"
    "sync"
)

type I1 interface {
    String() string
}

type I2 interface {
    PP() string
}

func PrintTwo[T I1, S I2](x T, y S, z string) {
    fmt.Printf("%s, %s, %s!!!\n", x.String(), y.PP(), z)
}

type T1 struct {
    Name string
}

func (t T1) String() string {
    return t.Name
}

type T2 struct {
```

```
    Name string
}

func (t T2) PP() string {
    return t.Name
}

func main() {
    t1 := T1{Name: "Vlad"}
    t2 := T2{Name: "Alex"}

    PrintTwo(t1, t2, "Hello")
}

// Vlad, Alex, Hello!!!
```

Пример:

```
package main

import (
    "fmt"
)

func [T comparable] Equal(a, b T) bool {
    return a == b
}

func main() {
  fmt.Println(Equal(1, 2), Equal(1, 1), Equal("", "a"), Equal("a", "a"))
}

// true, true, false, true
```

Пример с ошибкой:

```
func [T any] Equal(a, b T) bool {
    return a == b // Ошибка, так как для T any не определено сравнение через == и !=
}
```

Constraint для generics

Как уже выше написал, constraint - это интерфейс, который определяет ограничение для generics.

Из встроенных есть any и comparable.

Constraint интерфейс может иметь в себе type set, interface и методы.

Можно написать свой constraint интерфейс.

Допустим я хочу написать Sum(a, b T) для чисел, тогда мне нужно ограничить generics только для типов, которые можно сложить через '+':

```
package main

import "fmt"

type Number interface {
  int | int64 | float64
}

func Sum[T Number](a, b T) T {
  return a + b
}

func main() {
  fmt.Println(Sum(1, 2)) // 3
  fmt.Println(Sum(1.0, 2.0)) // 3.0
}
```

Конструкция int | int64 | float64 означает что в качестве типа можно использовать только эти типы.

Если в интерфейсе есть эта конструкция, то интерфейс становится constraint для generics и его теперь нельзя передавать в переменной.

Есть тонкий момент, что для кастомных типов, базовыми типами которых будут int, int64 и float64 функция выше не заработает, потому что type MyType int != int.

Для этого нужно писать так:

```
package main

import "fmt"

type Number interface {
  ~int | ~int64 | ~float64
}

type MyInt int

func Sum[T Number](a, b T) T {
  return a + b
}
```

```
func main() {
  fmt.Println(Sum(1, 2)) // 3
  fmt.Println(Sum(1.0, 2.0)) // 3.0
  fmt.Println(Sum(MyInt(1), MyInt(2))) // 3
}
```

~int означает, что в качестве типа можно использовать int и любой тип, базовый тип которого int.

Пример содержания в constraint и type set, и интерфейса и функций:

```
package main

import (
    "fmt"
    "sync"
)

type I1 interface {
    ~string | ~[]byte
    Print()
    fmt.Stringer
}

type MyType1 string

func (t MyType1) String() string {
    return string(t)
}

func (t MyType1) Print() {
    fmt.Println(t)
}

type I2 interface {
    Aloha() string
    String() string
}

type MyType2 struct {
    Name string
}

func (t MyType2) Aloha() string {
    return fmt.Sprintf("ALOHA: %s", t.Name)
}

func (t MyType2) String() string {
    return fmt.Sprintf("Aloha %s", t.Name)
}
```

```
func Test[T I1, S I2](a T, b S) string {
    return fmt.Sprintf("1: %s ; 2: %s\n", a, b)
}

func main() {
    x1 := MyType1("Hello")
    x2 := MyType2{Name: "Lilo"}
    fmt.Println(Test(x1, x2))
}
```

Пример функции, применяющей функцию к каждому элементу слайса:

```
package main

import "fmt"

func Map[T any, R any](items []T, f func(T) R) []R {
    result := make([]R, 0, len(items))

    for _, item := range items {
        result = append(result, f(item))
    }

    return result
}

func main() {
    nums := []int{1, 2, 3}

    strs := Map(nums, func(n int) string {
        return fmt.Sprintf("num=%d", n)
    })

    fmt.Println(strs) // [num=1 num=2 num=3]
}
```

Generic структура

Generic может быть структура:

```
// Обычная
type Box struct {
  Value int
}

// Generic структура
type Box[T any] struct {
```

```
  Value T
}
```

```
package main

import "fmt"

type Box[T any] struct {
    Value T
}

// Для методов нужно писать reciever с параметром типа
func (b Box[T]) GetValue() T {
    return b.Value
}

func (b *Box[T]) SetValue(value T) {
    b.Value = value
}

func main() {
    intBox := Box[int]{Value: 10}
    stringBox := Box[string]{Value: "hello"}

    fmt.Println(intBox.Value)    // 10
    fmt.Println(stringBox.Value) // hello
}
```

Классический пример - стек:

```
package main

import "fmt"

type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(value T) {
    s.items = append(s.items, value)
}

func (s *Stack[T]) Pop() (T, bool) {
    if len(s.items) == 0 {
        var zero T
        return zero, false
    }

    lastIndex := len(s.items) - 1
    value := s.items[lastIndex]
```

```
    s.items = s.items[:lastIndex]

    return value, true
}

func main() {
    var intStack Stack[int]

    intStack.Push(10)
    intStack.Push(20)

    v, ok := intStack.Pop()
    fmt.Println(v, ok) // 20 true

    v, ok = intStack.Pop()
    fmt.Println(v, ok) // 10 true

    v, ok = intStack.Pop()
    fmt.Println(v, ok) // 0 false
}
```

Здесь нужно обратить значение на var zero T. Так как мы не знаем, какой будет тип T, но все равно получим zero-value для этого типа.

Для структур лучше явно писать тип через b := Box[int]{Value: 10}.

Generic интерфейсы

Можно сделать generic interface:

```
type Repository[T any] interface {
    Save(value T) error
    FindByID(id int64) (T, error)
}
```

Например:

```
type User struct {
    ID   int64
    Name string
}

type UserRepository struct{}

func (UserRepository) Save(user User) error {
    return nil
}
```

```
func (UserRepository) FindByID(id int64) (User, error) {
    return User{ID: id, Name: "Bob"}, nil
}

// Теперь User реализует интерфейс Repository[User]

// Пример использования:
func ProcessUser(repo Repository[User]) {
    user, _ := repo.FindByID(1)
    fmt.Println(user)
}
```

Дженерики полезны при работе с одинаковыми алгоритмами для разных типов, коллекциями, утилитами и т.д.

Если нужно использовать отдельный тип, лучше использовать отдельный тип.

Если нужно написать какую-то свою мапу, кеш, ptr или другое, то лучше использовать generic.

Разница generic и interface

Generic обеспечивает типобезопасный код на этапе компиляции, а any = interface{} хранит значение любого типа при выполнении кода.

При этом если мы используем any, то нам придется использовать type assertion.

Пример:

```
func Double(x any) any {
    v := x.(int) // небезопасно, если не int -> panic
    return v * 2
}

func Double[T ~int | ~int64 | ~float64](x T) T {
    return x * 2 // все гуд
}
```

Горутины

Gorutines - это легковесный поток, который управляется средой выполнения Go.

```
go f(x, y, z) // старт новой горутины

go вызов_функции // старт новой горутины
```

Вычисление x, y, z происходит в текущей горутине, а выполнение f(x, y, z) происходит параллельно в новой горутине.

Горутины запускаются в одном адресном пространстве, поэтому shared memory нужно синхронизировать.

Примитивы синхронизации представлены в пакете sync.

Функция main в Go тоже горутина, когда компилятор достигает конца функции main - все горутины завершаются.

Каждая горутина представляет собой вызов функции и последовательно выполняет инструкции.

```
package main

import (
    "fmt"
    "time"
)

func say(s string) {
    for i := 0; i < 5; i++ {
        time.Sleep(100 * time.Millisecond)
        fmt.Println(s)
    }
}

func main() {
    go say("world")
    say("hello")
}
// Порядок не гарантирован
// hello
// world
// world
// hello
// ...
```

Контекст горутин

Каждая горутина выполняется в своем контексте, независимо от функции main.

То есть даже если мы по порядку запускаем несколько горутин, нельзя предсказать, в каком порядке они завершатся.

Синтаксис горутин

Для старта горутины используется ключевое слово `go` .

```
go SomeFunc()
```

или с анонимной функцией

```
go func(arg1, arg2, ...) {
  // code
} (arg1_. arg2_, ...)
```

Пример:

```
package main
import "fmt"

func main() {

    for i := 1; i < 7; i++{
        go func(n int){ // вызов анонимной функции
            result := 0
            for j := 1; j <= n; j++{
                result += j
            }
            fmt.Println(n, "-", result)
        }(i)
    }
    fmt.Scanln()        // ждем ввода пользователя
    fmt.Println("The End")
}


// ❯ go run .
// 5 - 15
// 6 - 21
// 3 - 6
// 4 - 10
// 1 - 1
// 2 - 3
// a
// The End
```

Sleep

Для работы со временем, таймерами, тикерами используется пакет time.

В пакете time есть функция Sleep(), которая приостанавливает выполнение горутины на указанное время.

```go
package main
import (
    "fmt"
    "time"
)

func main() {

    for i := 1; i < 7; i++{
        go sum(i)
    }
    time.Sleep(200 * time.Millisecond)
    fmt.Println("The End")
}

func sum(n int){
    result := 0
    for i := 1; i <= n; i++{
        result += i
    }
    fmt.Println(n, "-", result)
}

// ❯ go run .
// 6 - 21
// 1 - 1
// 2 - 3
// 3 - 6
// 4 - 10
// 5 - 15
// The End
```

Синхранизация горутин и WaitGroup

Для упрощения синхронизации горутин используется пакет *sync*.

Он в частности предоставляет структуру **sync.WaitGroup**, которая позволяет определить группу горутин, которые должны выполняться вместе как одна группа.

Можно установить блокировку, которая приостановит выполнение функции, пока вся группа горутин не завершит свое выполнение.

sync.WaitGroup - структура, которая содержит счетчик количества горутин, завершения которых нам нужно дождаться. Он используется для ожидания завершения всех горутин.

Если нужно передавать wg в другие функции, то нужно передавать указатель на него. Переиниализировать не рекомендуется (см документацию sync.WaitGroup).

Методы:

- Add: добавляет к счетчику число = сколько горутин нужно дождаться
- Done: вызывается после завершения каждой горутины, чтобы указать, что горутина завершена
- Wait: используется основной горутиной, ждет пока счетчик ожидаемых горутин станет 0, возвращается и продальшает выполнение основной функции.

Если коротко, то сначала создаем wg, добавляем в него горутины, вызываем их и прописываем defer wg.Done() внутри каждой, затем вызываем wg.Wait() и ждем завершения всех горутин.

Пример:

```
package main
import (
"fmt"
"sync"
"time"
)

func main() {
    var wg sync.WaitGroup
    wg.Add(2)       // в группе две горутины
    work := func(id int) {
        fmt.Printf("Горутина %d начала выполнение \n", id)
        time.Sleep(2 * time.Second)
        fmt.Printf("Горутина %d завершила выполнение \n", id)
        wg.Done() // сигнализируем, что горутина завершила работу
   }

   // вызываем горутины
   go work(1)
   go work(2)

   wg.Wait()        // ожидаем завершения обоих горутин
   fmt.Println("Горутины завершили выполнение")
}

// ❯ go run .
// Горутина 2 начала выполнение
// Горутина 1 начала выполнение
// Горутина 1 завершила выполнение
// Горутина 2 завершила выполнение
// Горутины завершили выполнение
```

Тут мы сделали анонимную функцию, она находится в области видимости основной горутины (в данном случае это main). Из-за этого мы можем обращаться к wg.

Пример:

```
package main
import (
"fmt"
"sync"
"time"
)

func work(id int, wg *sync.WaitGroup ) {

    fmt.Printf("Горутина %d начала выполнение \n", id)
    time.Sleep(2 * time.Second)  // имитация долгой работы горутины
    fmt.Printf("Горутина %d завершила выполнение \n", id)
    wg.Done()  // сигнализируем, что горутина завершила работу
}

func main() {
    var wg sync.WaitGroup
    wg.Add(2)       // в группе две горутины

   // вызываем горутины
   go work(1, &wg)
   go work(2, &wg)

   wg.Wait()        // ожидаем завершения обоих горутин
   fmt.Println("Горутины завершили выполнение")
}
```

Тут уже у нас не анонимная функция, нам нужно передать указатель на созданный wg.

Каналы

Каналы (channels) - инструмент коммуникации между горутинами.

В Go каналы имеют свой тип, который они могут передавать.

Каналы могут быть:

- буферизованные: в канале можно передавать несколько значений одновременно
- без буферизации: в канале можно передавать только один значение за раз

Deadlock - ситуация, когда все горутины блокируются и не могут продолжить работу.

Объявление канала:

```
var ch chan int
```

Zero-value: nil

Канал инициализируется через мункцию make():

```
ch1 := make(chan int)
var ch2 chan int = make(chan int)

ch3 := make(chan int, 10) // буферизованный канал с размером 10
```

Оператор канала <-

Для передачи данных в канал и для считывания из канала в Go существует оператор `<-` .

```
// Передача данных в канал
ch <- 10

// Считывание данных из канала
val := <-ch
```

Небуферизованные каналы

Небуферизованный канал - канал, в котором можно передавать только одно значение за раз.

Инициализируется через метод make() без передачи размера `ch := make(chan int)`

Если канал пустой, то горутина-получатель блокируется пока в канале не окажутся данные. Как только горутина-отправитель посылает данные в канал, горутина-получатель их вычитывает и продолжает работу.

Горутина-отправитель может отправлять данные только в пустой канал.

Горутина-отправитель блокируется, пока данные из канала не вычитают.

Пример:

```
package main

import (
    "fmt"
    "time"
)
```

```
func main() {
    ch := make(chan int)
    go SendNumbers(ch)
    for i := 0; i < 5; i++ {
        fmt.Printf("Got nummer: %d [%v]\n", <-ch, time.Now().Format(time.RFC3339)) // получате
    }

    fmt.Println("Done")
}

func SendNumbers(ch chan int) {
    for i := 0; i < 5; i++ {
        ch <- i // отправитель блокируется тут, пока данные не вычитают
        time.Sleep(time.Second * 3)
    }
}
// ❯ go run .
// Got nummer: 0 [2026-06-15T18:47:57+03:00]
// Got nummer: 1 [2026-06-15T18:48:00+03:00]
// Got nummer: 2 [2026-06-15T18:48:03+03:00]
// Got nummer: 3 [2026-06-15T18:48:06+03:00]
// Got nummer: 4 [2026-06-15T18:48:09+03:00]
// Done
```

Здесь мы в функции main читаем из канала переданное значение, а функция SendNumbers каждые 3 секунды отправляет на канал значение с индексом i.

Поэтому функция main их получает только раз в три секунды.

После того, как все данные получены (в данном случае мы 5 раз получаем значение), мы выходим из цикла, горутина получатель деблокируется и завершает работу.

Так как у нас функция не анонимная, то переменную канала нужно передать через параметр функции.

Можно переписать с анонимной функцией:

```
package main

import (
    "fmt"
    "time"
)

func main() {
  ch := make(chan int)

  go func() {
    for i := 0; i < 5; i++ {
```

```
      ch <- i // отправитель блокируется тут, пока данные не вычитают
      time.Sleep(time.Second * 3)
    }
  } ()

  for i := 0; i < 5; i++ {
    fmt.Printf("Got nummer: %d [%v]\n", <-ch, time.Now().Format(time.RFC3339)) // получатель б
  }

  fmt.Println("Done")
}
// ❯ go run .
// Got nummer: 0 [2026-06-16T13:51:07+03:00]
// Got nummer: 1 [2026-06-16T13:51:10+03:00]
// Got nummer: 2 [2026-06-16T13:51:13+03:00]
// Got nummer: 3 [2026-06-16T13:51:16+03:00]
// Got nummer: 4 [2026-06-16T13:51:19+03:00]
// Done
```

Здесь внутри горутины мы имеем доступ к переменной ch, поэтому можем взаимодействовать с ней без передачи ее через параметр.

Обмен данных через каналы

Если мы хотим обмениваться данными через каналы, то нужно чтобы одномоментно одна горутина отправляла данные в канал, а другая горутина получала данные из канала. Если мы определим отправление и получение данных через канал в самой функции main, то мы столкнемся с взаимоблокировкой.

Пример:

```
package main
import "fmt"

func main() {

    intCh := make(chan int)
    intCh <- 10      // функция main блокируется
    fmt.Println(<-intCh)
}
```

Но нужно отличать ситуацию, когда в одной горутине мы сначала кладем в канал, другая горутина получает данные, что-то с ними делает и потом кладет в этот же канал, после чего первая горутина их читает из того же канала. Тогда все будет норм.

```
package main
import "fmt"


func main() {

    intCh := make(chan int)

    go square(intCh)        // square ожидает получения через канал
    intCh <- 4              // отправляем в канал число
    fmt.Println("result := ", <-intCh)  // получаем из канала результат
    fmt.Println("The End")
}
// функция возведения в квадрат
func square(ch chan int){

    num := <-ch                 // получаем из канала число
    fmt.Println("num := ", num)
    ch <- num * num             // обратно отправляем квадрат числа
}
```

Тут мы сначала засунули число в канал, с ним что-то сделали, вернули в тот же канал, вычитали и вывели результат.

Буферизованные каналы

Буферизованный канал - это канал, в котором можно передавать несколько значений одновременно.

Инициализируется через метод make() с передачей размера `ch := make(chan int, 10)`

Если канал пуст, то горутина-получатель блокируется пока в канале не окажется хотя бы одно значение.

При отправке данных в канал горутина-отправитель блокируется, пока не освободится хотя бы одно место в канале.

Работает как очередь FIFO.

```
package main
import "fmt"

func main() {

    intCh := make(chan int, 3)
    intCh <- 10
    intCh <- 3
```

```
    intCh <- 24
    intCh <- 15  // блокировка - функция main ждет, когда освободится место в канале

    fmt.Println(<-intCh)
    fmt.Println("The End")
}
// Дедлок
```

```
package main

import "fmt"

func Receive(i int, ch chan int, resCh chan int) {
    for {
        val := <-ch
        if val == 24 {
            resCh <- i
        }
        fmt.Printf("R%d: %d\n", i, val)
    }
}

func Send(n, m int, ch chan int) {
    for i := n; i < m; i++ {
        ch <- i
    }
}

func main() {
    resCh := make(chan int)
    intCh := make(chan int, 10)
    go Receive(1, intCh, resCh)
    go Receive(2, intCh, resCh)

    j := 5
    for i := 0; i <= 20; i += 5 {
        go Send(i, j, intCh)
        j += 5
    }

    res := <-resCh
    fmt.Println("Result: ", res)

    fmt.Println("The End")
}
// Тут будет неопределенный вывод, но это будет работать
// Это связано с тем, что одна из горутин Recieve может обработать последний промежуток раньше
// ❯ go run .
// R1: 20
// R1: 21
// R1: 22
```

```
// R1: 23
// R1: 24
// Result:  1
// The End
// ❯ go run .
// R1: 20
// R1: 22
// R1: 23
// R1: 24
// R1: 5
// R2: 21
// R2: 7
// R2: 8
// R2: 9
// R2: 10
// R2: 11
// R2: 12
// R2: 13
// R2: 14
// R2: 15
// R2: 16
// R2: 17
// R2: 0
// R1: 6
// R1: 19
// Result:  1
// The End
```

Для буферизованных каналов:

- cap(): емкость канала
- len(): сколько элементов сейчас в канале

```
package main
import "fmt"

func main() {

    intCh := make(chan int, 3)
    intCh <- 10

    fmt.Println(cap(intCh))     // 3
    fmt.Println(len(intCh))     // 1

    fmt.Println(<-intCh)
}
```

Возвращение канала

Канал может быть возвращаемым значением функции, но нужно аккуратно подходить к записи и чтению.

```
package main
import "fmt"

func main() {
    fmt.Println("Start")
     // создание канала и получение из него данных
    fmt.Println(<-createChan(5)) // блокировка
    fmt.Println("End")
}
func createChan(n int) chan int{
    ch := make(chan int)    // создаем канал
    ch <- n      // отправляем данные в канал
    return ch   // возвращаем канал
}

// Тут будет дедлок, потому что на строке ch<-n заблокируется createChan
// return ch не выполнится
```

Собственно для того, чтобы определить функцию, которая возвращает канал, нужно чтобы все операции чтения-записи в канал выносились в отдельную горутину.

```
package main
import "fmt"

func main() {
    fmt.Println("Start")
     // создание канала и получение из него данных
    fmt.Println(<-createChan(5)) // 5
    fmt.Println("End")
}

func createChan(n int) chan int{
    ch := make(chan int)    // создаем канал
    go func(){
        ch <- n      // отправляем данные в канал
    }()             // запускаем горутину
    return ch   // возвращаем канал
}
```

Закрытие канала

После инициализации канал готов передавать данные и находится в открытом состоянии, пока его не закрыть.

Канал может быть закрыт через `close()` функцию (работает и с буферизованными, и с небуферизованными каналами).

Для горутин, которые ожидают значения из этого канала при закрытии будет 'сигнал', точнее они проснутся и вычитают (zero-value, false) из канала и деблокируются.

Горутина-отправитель закрывает канал, чтобы показать, что нет больше значений, которые будут посылаться в канал.

При получении значения горутина-получатель может использовать второе значение, полученное при чтении из канала, чтобы проверить, что канал открыт и в нем еще есть данные:

```
v, ok := <- ch
```

`ok` будет `false` если нет больше данных в канале и он закрыт.

```
go func() {
    defer close(ch)

    ch <- 1
    ch <- 2
    ch <- 3
}()
```

ВАЖНО!!!

Только отправитель должен закрывать канал, ни в коем случае не получатель. Запись в закрытый канал вызывает panic.

Каналы не как файлы, их не обязательно закрывать. Закрытие канала необходимо, если получателю точно нужно узнать, что больше не будет значений в канале, например, для завершения `range` цикла.

Читать из закрытого канала можно, вычитается zero-value.

Двунаправленные и однонаправленные каналы

По умолчанию через канал можно и отправлять и получать данные.

Но в Go можно определить канал, доступный только для отправки или только для чтения.

- только для отправки данных:

```
var inCh chan <-int
```

- только для чтения данных:

```
var outCh <-chan int
```

Это нужно чтобы зафиксировать типами намерения:

```
package main

import "fmt"

func producer(ch chan<- int) {
    defer close(ch)

    for i := 1; i <= 3; i++ {
        ch <- i
    }
}

func consumer(ch <-chan int) {
    for v := range ch {
        fmt.Println(v)
    }
}

func main() {
    ch := make(chan int)

    go producer(ch)

    consumer(ch)
}
```

Канал в принципе двунаправленный, но внутри функций consumer, producer его можно использовать только для чтения ли записи.

Теоретически можно создать однонаправленный канал, но когда мы запишем в него данные, то получить их будет невозможно и мы попадем в deadlock.

Синхронизация горутин через каналы

Для синхронизации горутин через каналы можно делать:

Канал с пустой структурой:

```
package main
import "fmt"

func main() {

   results := make(map[int]int)
   structCh := make(chan struct{})

   go square(5, structCh, results)

   val, ok := <-structCh        // ожидаем закрытия канала structCh
   fmt.Println(val, ok) // {} false

   for i, v := range results{
    fmt.Println(i, " - ", v)
   }
}

func square(n int, ch chan struct{}, results map[int]int){

    for i:=1; i <= n; i++{
        results[i] = i * i
    }
    close(ch)     // закрываем канал после завершения горутины
}
```

ВАЖНО!!!

Тут при close(ch) ничего не записывается в канал, а буквально слушающим горутинам приходит 'сигнал', что канал закрылся, они вычитывают из него zero-value и деблокируются.

Для ожидания нескольких горутин можно сделать done chan bool и вычитывать из него столько раз, сколько горутин ждем:

```
package main
import "fmt"

// вычисляем сумму первыз n чисел
func sum (n int, done chan bool){
    result := 0

    for k := 1; k <= n; k++{
        result += k
    }
    fmt.Println("Сумма первых", n, "чисел равна", result)

    done <- true  // уведомляем, что горутина равершила работу
}
```

```
func main() {

    done := make(chan bool)     // канал для ведомления о завершении горутины

    count := 4  // количество запускаемых горутин

    for i := 1; i <= count; i++{
        go sum(i+3, done)
    }

    // ожидаем завершения всех горутин
    for i := 1; i<= count; i++{
        <- done
    }
    fmt.Println("The End")
}
```

Получение значений канала в цикле

- `for v := range ch`

Для того, чтобы читать значения из канала до его закрытия, можно использовать конструкцию `for v := range ch` .

При использовании `for v := range ch` нужно обязательно закрывать канал отправителю, потому что иначе будет deadlock.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    ch := make(chan int)

    go func() {
        for i := 0; i < 5; i++ {
            ch <- i * i // отправляем в канал
            time.Sleep(time.Second)
        }

        close(ch) // закрываем канал, если забыть эту строку -> deadlock

        fmt.Println("Done")
    } ()
```

```
    for v := range ch {
        fmt.Println(v)
    }

    fmt.Println("The End")
}
// ❯ go run .
// 0
// 1
// 4
// 9
// 16
// Done
// The End
```

- select

Утверждение select дает возможность ждать несколько операций с каналами:

```
select {
  case v := <-ch1:
    fmt.Println("received from ch1:", v)
  case v := <-ch2:
    fmt.Println("received from ch2:", v)
}
```

Если ни один канал не готов, select блокируется.

Если готовы сразу несколько каналов, то Go выбирает псевдослучайно.

Select это специальная конструкция для горутин/каналов. Операторы case применяются к операциям отправки и получения значений в канал.

Синтаксис такой же, как у switch, за исключением того, что в качестве case у нас определение операции канала.

```
select{

    case операция_1:
        // обработка операции_1
    case операция_2:
        // обработка операции_2
.....................................
    case операция_N:
        // обработка операции_N
}
```

Пример:

```
package main

import "fmt"

func fibonacci(c, quit chan int) {
    x, y := 0, 1
    for {
        select {
        case c <- x:
            x, y = y, x+y
        case <-quit:
            fmt.Println("quit")
            return
        }
    }
}

func main() {
    c := make(chan int)
    quit := make(chan int)
    go func() {
        for i := 0; i < 10; i++ {
            fmt.Println(<-c)
        }
        quit <- 0
    }()
    fibonacci(c, quit)
}
// 0
// 1
// 1
// 2
// 3
// 5
// 8
// 13
// 21
// 34
// quit
```

Часть `default` в `select` выполняется, если ни один канал не готов:

```
select {
  case i := <-c:
    // use i
  default:
    // receiving from c would block
}
```

`default` используется чтобы попробовать получить или отправить значение без блокировки.

Нужно понимать, что select может за раз выполнить только одну ветку, поэтому если нужно повторно обрабатывать что-то или обрабатывать постоянно, то нужно использовать цикл for.

Пример:

```
package main

import (
    "fmt"
    "time"
)

func main() {
    start := time.Now()
    tick := time.Tick(100 * time.Millisecond)
    boom := time.After(500 * time.Millisecond)
    elapsed := func() time.Duration {
        return time.Since(start).Round(time.Millisecond)
    }
    for {
        select {
        case <-tick:
            fmt.Printf("[%6s] tick.\n", elapsed())
        case <-boom:
            fmt.Printf("[%6s] BOOM!\n", elapsed())
            return
        default:
            fmt.Printf("[%6s]     .\n", elapsed())
            time.Sleep(50 * time.Millisecond)
        }
    }
}
// [    0s]     .
// [  50ms]     .
// [ 100ms] tick.
// [ 100ms]     .
// [ 150ms]     .
// [ 200ms] tick.
// [ 200ms]     .
// [ 250ms]     .
// [ 300ms] tick.
// [ 300ms]     .
// [ 350ms]     .
// [ 400ms] tick.
// [ 400ms]     .
// [ 450ms]     .
// [ 500ms] BOOM!
```

Подходы использования каналов:

Каналы не всегда заменяют мьютексы.

Есть фраза:

```
Don't communicate by sharing memory; share memory by communicating.
```

То есть каналы хороши для:

- передачи событий
- pipeline
- worker pool
- сигналов завершения
- синхронизации этапов

Mutex хороши для:

- защиты общей map
- защиты счетчика
- кратких критических секций
- состояния объекта

Гонка данных и обнаружение

Гонка данных (data race) - ситуация, когда два и более потока одновременно обращаются к одному и тому же участку памяти и выполняют по крайней мере одну операцию записи.

Пример:

```go
package main

import (
"fmt"
"sync"
"time"
)

var wg sync.WaitGroup

var value = 22

func update_value(id int){

    fmt.Println("Goroutine", id, "starts")
    if(value==22){
        time.Sleep(1*time.Second)
        fmt.Println("Goroutine", id, "changes the value")
```

```
            value += 1
        }

        fmt.Println(value)
        wg.Done()       // сигнализируем, что горутина завершила выполнение
    }

    func main(){

        wg.Add(2)      // ждем 2 горутины

        go update_value(1)
        go update_value(2)

        wg.Wait()   // ожидаем завершения горутин
    }
```

Тут у нас могут быть такие выводы:

```
Goroutine 2 starts
Goroutine 2 changes the value
23
Goroutine 1 starts
23
```

или

```
Goroutine 2 starts
Goroutine 1 starts
Goroutine 1 changes the value
23
Goroutine 2 changes the value
24
```

В общем, неопределенное поведение, потому что первая горутина увеличит значение, а вторая может не успеть это увидеть и увеличить его еще раз.

Для проверки наличия гонки в Go есть флаг `-race`, который можно указать при сборке или запуске с `go run -race имя_программы` или `go build -race имя_программы`.

Тогда в терминал выведется сообщение с гонкой данных.

Использовать детектор гонок рекомендуется только в случае необходимости, так как он потребляет в 10 раз больше памяти и ресурсов процессора.

Mutex

Примитив синхронизации представленный в пакете sync.

Mutex - vutual exclusion (взаимное исключение).

Mutex - это структура данных, которая позволяет ограничить доступ к блоку кода для обработки нескольких потоков.

В Go стандартная библиотека предлагает `sync.Mutex` .

У него есть методы:

- `Lock()` : блокировка доступа
- `Unlock()` : разблокировка доступа

После `mu.Lock()` удобне писать `defer mu.Unlock()` .

Пример:

```
package main

import (
    "fmt"
    "sync"
    "time"
)

// SafeCounter is safe to use concurrently.
type SafeCounter struct {
    mu sync.Mutex
    v  map[string]int
}

// Inc increments the counter for the given key.
func (c *SafeCounter) Inc(key string) {
    c.mu.Lock()
    // Lock so only one goroutine at a time can access the map c.v.
    c.v[key]++
    c.mu.Unlock()
}

// Value returns the current value of the counter for the given key.
func (c *SafeCounter) Value(key string) int {
    c.mu.Lock()
    // Lock so only one goroutine at a time can access the map c.v.
    defer c.mu.Unlock()
    return c.v[key]
}

func main() {
    c := SafeCounter{v: make(map[string]int)}
    for i := 0; i < 1000; i++ {
```

```
        go c.Inc("somekey")
    }

    time.Sleep(time.Second)
    fmt.Println(c.Value("somekey"))
}
```

При блокировке мьютекса доступ коду получает горутина, которая его залочила. Остальные ждут, пока она не освободит блокировку.

ВАЖНО!!!

Нельзя копировать mutex после использования.

То есть методы, использующие mutex должны в качестве reciever принимать указатель на структуру с mutex, а не по значению.

Плохо:

```
type Counter struct {
    mu sync.Mutex
    n  int
}

func (c Counter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock()

    c.n++
}
```

Тут будет копирование всего `c Counter`, включая mutex.

Правильно:

```
type Counter struct {
    mu sync.Mutex
    n  int
}

func (c *Counter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock()

    c.n++
}
```

То есть структуры с mutex почти всегда используют через pointer reciever.

Иначе разные копии mutex будут блокировать разные данные.

Также лучше защищать опасные куски кода, а не весь метод.

Для чтения данных лучше возвращать копию, потому что можно вычитать например мапу, но вернуть ссылку на внутреннюю мап, которую внешний код может менять без защиты:

Плохо:

```
func (s *Store) Data() map[string]int {
    s.mu.RLock()
    defer s.mu.RUnlock()

    return s.data
}
```

Хорошо:

```
func (s *Store) Data() map[string]int {
    s.mu.RLock()
    defer s.mu.RUnlock()

    copyData := make(map[string]int, len(s.data))
    for k, v := range s.data {
        copyData[k] = v
    }

    return copyData
}
```

Аналогично со slices и другими типами (в которых под капотом header)

RWMutex

Примитив синхронизации представленный в пакете sync.

RWMutex - read-write mutex.

Он различает:

- читателей
- писателей

У него есть методы:

- `RLock()` : используется для получения блокировки на чтение. Несколько читателей могут получить эту блокировку в одной программе. Блокировка на чтение не привязан к определенному потоку.
- `RUnlock()` : освобождает полученную одну блокировку на чтение.
- `Lock()` : блокирует мьютекс для записи. Доступен только для одного потока за раз. Если другой поток получает блокировку для целей чтения/записи, Lock блокируется до тех пор, пока блокировка не станет доступной для получения.
- `Unlock()` : снимает блокировку на запись. При выполнении Unlock без получения блокировки мы получим ошибку во время выполнения.

Если попроще:

- `RLock()` : я хочу только читать
- `Lock()` : я хочу только писать/изменять

Главная идея:

`Mutex` думает так:

Внутрь может зайти только одна горутина и не важно, она читает или пишет.

`RWMutex` думает так:

- Читать одновременно могут многие
- Писать может только одна
- Пока кто-то пишет - никто не читает
- Пока кто-то читает - никто не пишет

То есть RWMutex полезен, когда много чтений и мало записей.

RLock():

`RLock()` - зайти как читатель. Если сейчас никто не пишет, то читатель заходит.

Читателей при этом может быть много и все могут читать одновременно.

```
goroutine 1: RLock() -> вошла
goroutine 2: RLock() -> вошла
goroutine 3: RLock() -> вошла
```

Пример:

```
mu.RLock()
value := data["name"]
```

```
mu.RUnlock()
```

RUnlock():

`RUnlock()` - выйти как читатель. То есть я закончил читать. Не используется без `RLock()`.

Lock():

`Lock()` - зайти как писатель. То есть я хочу писать/изменять данные.

Писатель может зайти только если:

- нет активных читателей
- нет активных писателей

То есть `Lock()` ждет, пока все читатели и писатели выйдут.

Пример:

```
mu.Lock()
data["name"] = "Vlad"
mu.Unlock()
```

Unlock():

`Unlock()` - выйти как писатель. То есть я закончил писать/изменять данные.

Каждому `Unlock()` должен соответствовать `Lock()`.

Таблица блокировок RWMutex:

Сейчас внутри	Новый RLock()	Новый Lock()
никого нет	вошла	вошла
читатели есть	вошла	ждет
писатели есть	ждет	ждет

То есть:

- Если уже есть читатели:

```
RLock + RLock = можно
RLock + Lock  = нельзя, писатель ждет
```

- Если уже есть писатель:

```
Lock + RLock = нельзя, читатель ждет
Lock + Lock  = нельзя, второй писатель ждет
```

Пример:

```go
package localcache

import "sync"

type LocalCache[T comparable, V any] struct {
    data map[T]V
    mu   sync.RWMutex
}

// Create a new safe map
//
// Usage: m := safemap.New[keyType, valueType]()
func New[T comparable, V any]() *LocalCache[T, V] {
    return &LocalCache[T, V]{data: make(map[T]V)}
}

// Set a value in the map
//
// Usage: m.Set(key, value)
func (m *LocalCache[T, V]) Set(key T, value V) {
    m.mu.Lock()
    defer m.mu.Unlock()

    if m.data == nil {
        m.data = make(map[T]V)
    }

    m.data[key] = value
}

// Get a value from the map by key
//
// Usage: val, ok := m.Get(key)
func (m *LocalCache[T, V]) Get(key T) (V, bool) {
    m.mu.RLock()
    defer m.mu.RUnlock()

    val, ok := m.data[key]

    return val, ok
}

// Delete a value from the map by key
//
// Usage: m.Delete(key)
```

```
func (m *LocalCache[T, V]) Delete(key T) {
    m.mu.Lock()
    defer m.mu.Unlock()

    delete(m.data, key)
}

// Clear the map
//
// Usage: m.Clear()
func (m *LocalCache[T, V]) Clear() {
    m.mu.Lock()
    defer m.mu.Unlock()

    m.data = make(map[T]V)
}
```

Нельзя

Нельзя даунгрейдить Lock() до RLock() напрямую:

```
mu.Lock()
// изменили данные
mu.Unlock()

mu.RLock()
// читаем
mu.RUnlock()
```

Но нужно учитывать, что между mu.Unlock() и mu.RLock() может зайти другая горутина

Нельзя апгрейдить RLock() до Lock():

```
mu.RLock()

if needWrite {
    mu.Lock() // deadlock/проблема
}

mu.RUnlock()
```

Тут может быть такая проблема, что мы вначале зашли как читатель и залочили RLock(), потом мы говорим, что мы хотим еще и записать.

Но Lock() ждет, пока читатели не выйдут, а так как мы и есть читатели и еще залочили Lock(), то мы попали в Deadlock.

Кароче нужно четко сначала определиться меняются или читаются данные, могут ли они между Lock и Unlock поменяться и т.д.

Правильно было бы так:

```
mu.RLock()
needWrite := checkSomething()
mu.RUnlock()

if needWrite {
    mu.Lock()
    // перед записью лучше перепроверить условие
    mu.Unlock()
}
```

Но тут тоже между mu.RUnlock() и mu.Lock() другая горутина могла зайти и поменять данные.

Еще раз про блокировки:

```
RLock блокирует Lock.
Lock блокирует RLock.
Lock блокирует Lock.
RLock НЕ блокирует другой RLock.

То есть:

читатель не мешает читателю
читатель мешает писателю
писатель мешает читателю
писатель мешает писателю
```

Обработка ошибок

В Go программы выражают ошибку выполнения с помощью типа `error`.

`error` - встроенный интерфейс.

Для реализации интерфейса `error` требуется реализовать метод `Error() string`.

В стандартной библиотеке Go есть пакет `errors`, который реализует множество функций для работы с ошибками.

Ошибка - просто значение, которое нужно обработать. При получении ошибки программа не завершается (если не кинули panic).

errors.New()

`errors.New(string)` - возвращает значение конкретного типа, который реализирует `error` интерфейс, который содержит только текстовое представление ошибки.

Под капотом там структура:

```
// errors/errors.go
package errors

// New returns an error that formats as the given text.
// Each call to New returns a distinct error value even if the text is identical.
func New(text string) error {
    return &errorString{text}
}

// errorString is a trivial implementation of error.
type errorString struct {
    s string
}

func (e *errorString) Error() string {
    return e.s
}
```

Wrapping

Wrapping - заворачивание ошибки в другую ошибку.

Wrapping нужен для сохранения контекста ошибки и сборки цепочки ошибок.

Для того, чтобы завернуть ошибку чаще всего изпользуют `fmt.Errorf("description: %w", err)` .

`%w` - wrap

Модели завертывания ошибок

- цепочка ошибок: последовательное заворачивание ошибок

При использовании последовательного заворачивания, будет создаваться новая ошибка с методом `Unwrap() error` ,

по которому errors.Is и errors.As будут ходить по цепочке.

```
// Пусть это код data layer
// db.go
package db
```

```go
import (
  "errors"
  "database/sql"
)

var ErrNotFound = errors.New("db: not found")

func (db *DB) GetUser(id int) (User, error) {
  user, err := db.ExecQuery("SELECT * FROM users WHERE id = ?", id)
  //...
  if err != nil {
    if errors.Is(err, sql.ErrNoRows) {
      // тут мы через %v ошибку sql.ErrNoRows просто форматируем как строку
      // заворачиваем через %w ошибку db.ErrNotFound
      return nil, fmt.Errorf("%w: user not found: %v", ErrNotFound, err)
    }
  }

  return user, nil
}

// Пусть это код service layer
// service.go
package service

import (
  "errors"
  "db"
)

var ErrNotFound = errors.New("service: not found")

func (s *Service) GetUser(id int) (User, error) {
  user, err := db.GetUser(id)
  if err != nil {
    if errors.Is(err, db.ErrNotFound) {
      // Тут мы заворачиваем ошибку service.ErrNotFound и оставляем полученную ошибку как текс
      return nil, fmt.Errorf("%w: error getting user: %v", ErrNotFound, err)
    }

    return nil, err
  }

  return user, nil
}

// Пусть это код server layer
// server.go
package server

import (
  "errors"
  "service"
```

```
)

func (h *Handler) GetUserHandler(w http.ResponseWriter, r *http.Request) {
  // тут допустим мы запрос заанмаршалили
  // req, err := json.NewDecoder(r.Body).Decode(&req)
  // ...

  resp, err := service.GetUser(req, id)
  if err != nil {
    if errors.Is(err, service.ErrNotFound) {
      // errors.Is пройдется по цепочке ошибок, найдет service.ErrNotFound
      // и если мы нашли, то проставим нужный статус код
      http.Error(w, err.Error(), http.StatusNotFound)
      return
    }

    http.Error(w, err.Error(), http.StatusInternalServerError)
  }
}
```

Тут у нас последовательно заворачиваются ошибки, цепочка будет такая:

```
# Ошибка, дошедшая до сервер слоя:
service: not found: error getting user: db: not found: sql: no rows in result set
# Ошибка, дошедшая до сервис слоя:
db: not found: sql: no rows in result set
# Ошибка дата слоя
sql: no rows in result set
```

Если мы развернем ошибку, то будет так:

```
errUwrapped := errors.Unwrap(err) // db: not found: sql: no rows in result set
errUnwrapped := errors.Unwrap(errUwrapped) // nil, потому что мы sql.ErrNoRows не завернули, а
```

или более простой пример:

```
package main

import (
    "errors"
    "fmt"
)

var (
    Err1 error = errors.New("error1")
    Err2 error = errors.New("error2")
    Err3 error = errors.New("error3")
)
```

```
type CustomErr struct {
    msg string
}

func (e *CustomErr) Error() string {
    return e.msg
}

func main() {
    err := fmt.Errorf("%w", Err1)
    err = fmt.Errorf("%w:%v", Err2, err)
    err = fmt.Errorf("%w:%v", Err3, err)
    fmt.Printf("%v\n\n", err)
    fmt.Printf("%v\n", errors.Unwrap(err))
    fmt.Printf("%v\n", errors.Unwrap(errors.Unwrap(err)))
}
// ❯ go run .
// error3:error2:error1

// error3
// <nil>
```

Но тут нужно запомнить, что если мы бы завернули множественно через %w, то Unwrap() уже работал бы с []error.

Тогда у нас первый результать Unwrap() уже был бы nil.

```
package main

import (
    "errors"
    "fmt"
)

var (
    Err1 error = errors.New("error1")
    Err2 error = errors.New("error2")
    Err3 error = errors.New("error3")
)

type CustomErr struct {
    msg string
}

func (e *CustomErr) Error() string {
    return e.msg
}

func main() {
    err := fmt.Errorf("%w", Err1)
```

```
    err = fmt.Errorf("%w:%w", Err2, err)
    err = fmt.Errorf("%w:%w", Err3, err)
    fmt.Printf("%v\n\n", err)
    fmt.Printf("%v\n", errors.Unwrap(err))
    fmt.Printf("%v\n", errors.Unwrap(errors.Unwrap(err)))
}
// ❯ go run .
// error3:error2:error1

// <nil>
// <nil>
```

- дерево ошибок: получаем при использовании errors.Join().

То есть при использовании errors.Join() мы получаем ошибку с методом `Unwrap() []error`,

который будет содержать все ошибки на одном уровне.

Используется, когда мы хотим, чтобы новая ошибка была собрана из нескольких независимых ошибок.

```
// Пример из головы:
// Допустим мы в мидлваре проверяем доступ пользака к ручке и не находим его

var ErrNotFound = errors.New("not found")
var ErrUnauthorized = errors.New("unauthorized")

func (m *AuthMiddleware) CheckAccess(ctx context.Context, userID int) error {
  user, err := m.db.GetUser(userID)
  if err != nil {
    if errors.Is(err, sql.ErrNoRows) {
      return errors.Join(ErrNotFound, ErrUnauthorized)
    }
  }

  // код проверок
  //...

  return nil
}


func (s *Server) authMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        header := r.Header.Get("Authorization")
        if header == "" {
            s.logger.Error("error getting header", slog.String("error", ErrEmptyAuthHeader.Err
            writeErrorResponse(w, constants.ErrBadRequest, http.StatusUnauthorized, "empty req
            return
        }
```

```
        token, ok := strings.CutPrefix(header, "Bearer ")
        if !ok {
            s.logger.Error("error getting token string", slog.String("error", ErrInternal.Erro
            writeErrorResponse(w, constants.ErrBadRequest, http.StatusUnauthorized, "empty req
            return
        }

        claims, err := s.validateToken(token)
        if err != nil {
            s.logger.Error("error validating token", slog.String("error", err.Error()))
            writeErrorResponse(w, err, http.StatusUnauthorized)
            return
        }

    if err := checkUserAccess(ctx, claims.UserID); err != nil {
      // Вот тут я допустим захочу проверить, что мне вернулось
      // Допустим я хочу сначала проверить на NotFound, а потом проверить на Unauthorized
      // Потому что Unauthorized я могу вернуть при любой другой ошибке,
      // а NotFound я хочу вернуть отдельно
      // !!! КАРОЧЕ ТЕХНИЧЕСКИ НОРМ, НО ИМЕННО ЭТОТ КЕЙС ВОЗДУШНЫЙ
      // !!! ПОТОМУ ЧТО ЛУЧШЕ ВЫБИРАТЬ ОДНУ ОСНОВНУЮ, А ОСТАЛЬНЫЕ ДЛЯ ПОНИМАНИЯ КОНТЕКСТА ОШИБ
      if errors.Is(err, ErrNotFound) {
        writeErrorResponse(w, "user not found", http.StatusNotFound)
        return
      }

      s.logger.Error("error checking user access", slog.String("error", err.Error()))
      writeErrorResponse(w, err, http.StatusUnauthorized)
      return
    }

        r.Header.Set("X-User-ID", claims.UserId)
        r.Header.Set("X-Dormitory-ID", claims.DormitoryId)

        s.logger.Debug("extracted ids from token", slog.String("userId", claims.UserId), slog.

        next.ServeHTTP(w, r)
    })
}
```

При этом если я итоговую ошибку буду разворачивать, то будет так:

```
# Ошибка, дошедшая до мидлвари:
not found
unauthorized
# Тут дошла имеено ошибка, содержащая внутри []error

# Далее если я сделаю unwrap на ошибке из []error, то получим такой результат:
<nil>
```

Если более простой пример:

```
package main

import (
    "errors"
    "fmt"
)

var (
    Err1 error = errors.New("error1")
    Err2 error = errors.New("error2")
)

type CustomErr struct {
    msg string
}

func (e *CustomErr) Error() string {
    return e.msg
}

func main() {
    err := errors.Join(Err1, Err2)
    fmt.Printf("%v\n\n", err)
    fmt.Printf("%v\n", errors.Unwrap(err))
}
// ❯ go run .
// error1
// error2
//
// <nil>
```

Unwrapping

Unwrapping - развертывание ошибки.

Unwrapping нужен чтобы развернуть цепочку ошибок и достать ошибку предыдущего уровня.

`Unwrap()` используется методами errors.Is() и errors.As() для обхода по цепочке или дереву ошибок.

В зависимости от выбранного метода заворачивания:

- Цепочка: `Unwrap() error`
- Дерево: `Unwrap() []error`

Unwrap() возвращает первую завернутую ошибку предыдущего уровня.

Или если по другому: следующую вложенную ошибку, если у `err` есть метод `Unwrap() error`.

```
package main

import (
    "errors"
    "fmt"
)

var (
    Err1 error = errors.New("error1")
    Err2 error = errors.New("error2")
    Err3 error = errors.New("error3")
)

func main() {
  err := fmt.Errorf("%w", Err1)
  err = fmt.Errorf("%w:%v", Err2, err)

  fmt.Printf("%v\n\n", err)
  fmt.Printf("%v\n", errors.Unwrap(err))
  fmt.Printf("%v\n", errors.Unwrap(errors.Unwrap(err)))
}

// ❯ go run .
// error2:error1

// error2
// <nil>
```

Тут важно понимать, что Unwrap() смотрит на ошибку уровня ниже, но если мы сделаем что-то типа:

```
err := errors.Join(Err1, Err2)
// или
err := fmt.Errorf("%w: %w", Err1, Err2)
```

То `Unwrap()` вернет `<nil>`, так как просто `Unwrap()` не в контесте errors.Is и errors.As умеет работать только с `error`, а не `[]error`.

Вообще отдельно `Unwrap()` не используют обычно, лучше использовать errors.Is и errors.As.

errors.Is() и errors.As()

errors.Is() и errors.As() позволяют пройтись по цепочке/дереву ошибок (рекурсивно внутрь) и проверить наличие в нем определенной ошибки.

```
package main

import (
    "errors"
    "fmt"
)

type CustomErr struct {
    msg string
    source string
}

func (e *CustomErr) Error() string {
    return fmt.Sprintf("source=%s msg=%s", e.source, e.msg)
}

var (
    Err1 error = errors.New("error1")
    Err2 error = errors.New("error2")
    Err3 error = errors.New("error3")
)

func main() {
    custErr := &CustomErr{msg: "custom error", source: "app"}
  err := fmt.Errorf("wrapped custom: %w", custErr)
    err = errors.Join(fmt.Errorf("wrapped double custom: %w", err), Err1)

    fmt.Printf("%v\n\n", err)

    var custErr2 *CustomErr
    if errors.As(err, &custErr2) {
        fmt.Printf("custErr2=%v\n", custErr2)
    } else {
        fmt.Printf("not custErr2: %v\n", err)
    }
}
// ❯ go run .
// wrapped double custom: wrapped custom: source=app msg=custom error
// error1

// custErr2=source=app msg=custom error
```

Тут у нас `errors.As` прошелся внутрь на два слоя и нашел ошибку типа `CustomErr`.

На первом шаге он посмотрел на дерево ошибок, так как у нас ошибка `err` после `errors.Join()` такая:

```
err-|
    |- wrapped double custom: wrapped custom: source=app msg=custom error
    |           |- wrapped custom: source=app msg=custom error
```

```
    |                   |- source=app msg=custom error
    |
    |- error1
```

Жожем также проверить с errors.ls:

```
package main

import (
    "errors"
    "fmt"
)

type CustomErr struct {
    msg string
    source string
}

func (e *CustomErr) Error() string {
    return fmt.Sprintf("source=%s msg=%s", e.source, e.msg)
}

var (
    Err1 error = errors.New("error1")
    Err2 error = errors.New("error2")
    Err3 error = errors.New("error3")
)

func main() {
    custErr := &CustomErr{msg: "custom error", source: "app"}
  err := fmt.Errorf("wrapped custom: %w", custErr)
    err = errors.Join(fmt.Errorf("wrapped double custom: %w", err), Err1)

    fmt.Printf("%v\n\n", err)

    fmt.Println(errors.Is(err, Err1))
    fmt.Println(errors.Is(err, custErr))
    fmt.Println(errors.Is(err, Err2))

    fmt.Println()

    err = fmt.Errorf("%w: %v", Err2, err)
    fmt.Println(errors.Is(err, Err1))
    fmt.Println(errors.Is(err, custErr))
    fmt.Println(errors.Is(err, Err2))

  fmt.Println()
  fmt.Println(errors.Is(err, &CustomErr{msg: "custom error", source: "app"}))

}
// ❯ go run .
```

```
// wrapped double custom: wrapped custom: source=app msg=custom error
// error1

// true
// true
// false

// false
// false
// true

// false
```

Тут у нас до момента, пока мы не завернули `err` как текст через `%v`, `errors.Is` нам отдал, что в `err` есть `Err1` и `CustomErr`.

После заворачивания через `%v` и обертывания ошибки в `Err2` через `%w` `errors.Is` нам не отдал, что в `err` есть только `Err2`.

Также у нас `fmt.Println(errors.Is(err, custErr)) //true` потому что мы сравнили с той же ошибкой, которую положили.

Ниже мы сделали `fmt.Println(errors.Is(err, &CustomErr{msg: "custom error", source: "app"})) //false`, потому что это уже другая созданная ошибка, а не тип.

Если подрезюмировать, то `errors.Is` чаще используют для sentinel errors (те, которые мы сами отдельно объявили), а `errors.As` используют для проверки кастомной ошибки или специфичной и присваивания ее в отдельную переменную.

Пример:

```
func (c *Database) register(
    ctx context.Context,
    driver Driver,
    request *dbtypes.RegisterRequest,
) (*dbtypes.RegisterResponse, error) {
    if request == nil {
        return nil, dberrors.ErrBadRequest
    }

    var (
        psql = squirrel.StatementBuilder.PlaceholderFormat(squirrel.Dollar)

        usersTable = fmt.Sprintf("%s.%s", constants.SchemaName, constants.UsersTableName)
    )

    uid := uuid.NewString()

    queryBuilder := psql.Insert(usersTable).
```

```
            Columns(
                "id", "email", "password", "dormitory_id",
            ).
            Values(
                uid, request.Email, request.Password, request.DormitoryId,
            )

        query, args, err := queryBuilder.ToSql()
        if err != nil {
            return nil, fmt.Errorf("%w: error building register query: %v", dberrors.ErrInternal,
        }
        _, err = driver.ExecContext(ctx, query, args...)
        if err != nil {
            var pgErr *pq.Error // Тут мы хотим проверить, пришла ли ошибка тпиа pq.Error и если ч
            if errors.As(err, &pgErr) && pgErr.Code == dberrors.PGErrUniqueViolation {
                return nil, fmt.Errorf("%w: error user with same credentials exists: %v", dberrors
            }

            return nil, fmt.Errorf("%w: error executing register query: %v", dberrors.ErrInternal,
        }

        return &dbtypes.RegisterResponse{
            UserId:      uid,
            DormitoryId: request.DormitoryId,
        }, nil
    }
```

panic(), defer, recover()

`panic()` - механизм аварийного завершения текущего потока выполнения в горутине.

Он используется для проверки непредвиденных ошибок и похож на exception в других ЯП.

Вызов panic может произойти при двух сценариях:

- мы сами вызвали панику `panic(err)`
- Runtime вызвал panic из кода программы

Runtime чаще всего может вызвать panic если:

- пишем в nil map/slice (если не через append(), а через s[k] = v)
- nil dereference / забыли проинициализировать структуру и пишем в ее поле или читаем ее поле
- деление на 0 (для целых типов, для float64 паники не будет: 10.0/0.0 = +Inf)
- выход за предели slice

При срабатывании `panic()` в функции останавливается выполнение этой функции и завершается выполнение программы.

Если в функции, где произошла ошибка, вызываются функции с оператором `defer`, то перед завершением программы выполняются они (именно текущей горутины).

`panic()` принимает один аргумент типа `interface{}`, который может быть любым типом.

Если кратко: `panic()` завершает текущую горутину, возвращает не нулевой exit code (если не было recover) и перед заверешением всех функций выполняет defer этой горутины.

То есть если мы пойдем по стеку вызовов и наверху будет main, то вся программа упадет. Если не main, то упадет горутина, которая вызвала panic.

Порядок выполнения такой:

```
1. текущая функция останавливает обычное выполнение
2. выполняются ее defer'ы
3. panic поднимается в вызывающую функцию
4. там тоже выполняются defer'ы
5. panic идет выше по стеку
6. если никто не сделал recover — программа падает
```

Это называется stack unwinding.

Пример:

```
package main
import "fmt"

func main() {
    fmt.Println(divide(15, 5))
    fmt.Println(divide(4, 0))
    fmt.Println("Program has been finished")
}
func divide(x, y float64) float64{
    if y == 0{
        panic("Division by zero!")
    }
    return x / y
}
// ❯ go run .
// 3
// panic: Division by zero!

// goroutine 1 [running]:
// main.divide(...)
//         /home/vlad/GOshka/Tour/go-tour/main.go:11
// main.main()
//         /home/vlad/GOshka/Tour/go-tour/main.go:6 +0x6a
// exit status 2
```

Пример с defer:

```
package main
import "fmt"

func mainDefer() {
    fmt.Println("main defer executed")
}

func divideDefer(x, y float64) {
    fmt.Printf("divide defer executed: x=%v y=%v\n", x, y)
}

func main() {
    defer mainDefer()
    fmt.Println(divide(15, 5))
    fmt.Println(divide(4, 0))
    fmt.Println("Program has been finished")
}
func divide(x, y float64) float64{
    defer divideDefer(x, y)
    if y == 0{
        panic("Division by zero!")
    }
    return x / y
}
// ❯ go run .
// divide defer executed: x=15 y=5
// 3
// divide defer executed: x=4 y=0
// main defer executed
// panic: Division by zero!

// goroutine 1 [running]:
// main.divide(0x4e57d8?, 0xc000088018?)
//         /home/vlad/GOshka/Tour/go-tour/main.go:21 +0x8e
// main.main()
//         /home/vlad/GOshka/Tour/go-tour/main.go:15 +0x96
// exit status 2
```

Еще пример с defer:

```
package main

import "fmt"

func a() {
    defer fmt.Println("defer a")
    panic("boom")
}
```

```go
func b() {
    defer fmt.Println("defer b")
    a()
}

func c() {
    defer fmt.Println("defer c")
    b()
}

func main() {
    defer fmt.Println("defer main")
    c()
}
// ❯ go run .
// defer a
// defer b
// defer c
// defer main
// panic: boom

// goroutine 1 [running]:
// main.a()
//         /home/vlad/GOshka/Tour/go-tour/main.go:7 +0x59
// main.b()
//         /home/vlad/GOshka/Tour/go-tour/main.go:12 +0x52
// main.c()
//         /home/vlad/GOshka/Tour/go-tour/main.go:17 +0x52
// main.main()
//         /home/vlad/GOshka/Tour/go-tour/main.go:22 +0x52
// exit status 2
```

`panic` используют для:

- невозможных состояний
- ошибок программиста
- нарушения инвариантов
- критической ошибки инициализации

recover():

`recover()` - механизм отлавливания panic. То есть recover() останавливает распространение panic, если вызван внутри deferred-функции в той же горутине.

если мы считаем значение recover(), то в ней будет то, что мы положили как аргумент panic.

ВАЖНО!!!

- recover() должен быть !!!в defer!!! той горутины, в которой мы хотим отлавливать panic. Recover работает только в defer. Если мы сделали recover() в main, а упала другая горутина, то main не отловит эту panic и все равно упадет вся программа.
- сам recover() ничего не перезапускает, то есть если у меня лег воркер, то recover() должен быть такой:

```
func (p *WorkerPool) startWorker(ctx context.Context, workerID int) {
  defer func() {
    if r := recover(); r != nil {
      p.logger.Error("Worker panic recovered - restarting",
                slog.Int("worker_id", workerID),
                slog.Any("panic", r))

            time.Sleep(constants.RecoverDelay)

      p.startWorker(ctx, workerID) // но лучше это делать не рекурсивно, иначе накопится стек
    }
  } ()

  for {
        select {
        case batch := <-p.batchQueue:
            p.processBatch(ctx, batch, workerID)
        case <-p.stopChan:
            p.logger.Info("Worker stopped", slog.Int("worker_id", workerID))

            return

        case <-ctx.Done():
            p.logger.Info("Worker context done", slog.Int("worker_id", workerID))

            return

        }
    }
}
```

- recover() не возвращает на строку выполнения после panic(), а просто безопасно выходит из функций по стеку вызовов и возвращает zero-value, если ничего другого не вернули явно после recover().
- если паника не была отловлена в той же горутине, падает вся программа. То есть unhandled panic => вся программа упала.

```
package main

import "fmt"
func divide(x, y float64) float64{
```

```
    defer func() {
        if r := recover(); r != nil { // в r будет лежать "Division by zero!"
            fmt.Println("recovered:", r)
        }
    }()
    if y == 0{
        panic("Division by zero!")
                fmt.Println("AFTER PANIC") // недостижимая строка, никогда не выполнится
    }

    return x / y
}

func main() {
  fmt.Println(divide(10, 0))
    fmt.Println(divide(15, 1))
  fmt.Println("Program done")
}
// ❯ go run .
// recovered: Division by zero!
// 0
// 15
// Program done
```

Контекст

`context.Context` - интерфейс:

```
type Context interface {
    Deadline() (deadline time.Time, ok bool)
    Done() <-chan struct{}
    Err() error
    Value(key any) any
}
```

- Deadline() - возвращает время, когда контекст должен быть отменен, если он есть.
- Done() - возвращает канал для остановки, который закрывается когда вызывается cancel() или по истечении timeout и т.д. Обычно используется в select
- Err() - возвращает ошибку: если Done закрыт, то возвращает non-nil ошибку с описанием причины. Если Done еще ready, вернет nil
- Value() - возвращает значение контекста по ключу, если его нет, то возвращает nil.

Лучше про сам интерфейс почитать в документации

То есть `ctx context.Context` это значение типа, который реализует интерфейс `context.Context`.

Контексты с deadline, timeout, cancel и value создаются от родительского контекста.

Структура взаимосвязи контекстов - дерево:

```
parent ctx
  ├── child ctx
  │   ├── grandchild ctx
  │   └── grandchild ctx
  └── child ctx
```

То есть если отменить контекст родителя, то отменятся все контесты дети.

При этом если я создал родительский пустой контекст, затем создал от него контекст ребенок со значением или др. то у родителя не будет этих значений.

Также если отменю ребенка, то родительский контекст не отменится.

`cancel()` закрывает канал ctx.Done().

context.Background

`context.Background() context.Context` - создает пустой контекст без cancel, timeout, deadline и value.

Тут создается пустой контекст без cancel, timeout, deadline и value:

```
// context.Background() context.Context
ctx := context.Background()
```

ВАЖНО!!!

Если в функцию уже пришел контекст, то заменять его нельзя, так как мы потеряем значения из родительского контекста.

```
func SomeFunc(ctx context.Context) {
    ctx = context.Background() // плохо, так как потеряли value, timeout, cancel и т.д.

  res, err := SomeService.DoSomething(ctx) // тут уже пустой контекст
  // code...
}
```

context.TODO

`context.TODO() context.Context` - создает пустой контекст без cancel, timeout, deadline и value.

Похож на context.Background, используется когда еще не понятно, какой контекст нужно использовать.

Лучше в prod коде не использовать.

context.WithValue

`context.WithValue(parent context.Context, key, val any) context.Context` - создать контекст с одним значением типа value:

Тут мы создаем пустой контекст (можно создать от переданного в функцию).

Затем мы от него создаем контест ребенка с значением "value" по ключу CtxKeyUser:

```
// context.WithValue(parent context.Context, key, val any) context.Context
type CtxKey string

const (
  CtxKeyUser CtxKey = "user"
)

func main() {
  ctx := context.Background()
  ctx = context.WithValue(ctx, CtxKeyUser, "value")
}
```

Тут важно, что context.WithValue принимает в качестве ключа и значения тип `any`, поэтому чтобы достать само значение, нужен `type assertion`:

```
package main

import (
    "context"
    "fmt"
)

// context.WithValue(parent context.Context, key, val any) context.Context
type CtxKey string

const (
    CtxKeyUser CtxKey = "user"
)

func main() {
    ctx := context.Background()
    ctx = context.WithValue(ctx, CtxKeyUser, "value")
```

```
    if val, ok := ctx.Value(CtxKeyUser).(int); ok {
        fmt.Printf("value is int: %v %t\n", val, ok) // value true
    }

    if val, ok := ctx.Value(CtxKeyUser).(string); ok {
        fmt.Printf("value is string: %v %t\n", val, ok) // value is string: value true
    }
}
// ❯ go run .
// value is string: value true
```

Для получения значения используется метод `Value(key any) any`, который возвращает значение типа `any`, если по ключу есть значение и `nil`, если значения по ключу в контексте нет.

```
package main

import (
    "context"
    "fmt"
)

// context.WithValue(parent context.Context, key, val any) context.Context
type CtxKey string

const (
    CtxKeyUser CtxKey = "user"
)

func main() {
    ctxBase := context.Background()
    ctx := context.WithValue(ctxBase, CtxKeyUser, "value")

    if val, ok := ctx.Value(CtxKeyUser).(int); ok {
        fmt.Printf("value is int: %v %t\n", val, ok) // value true
    }

    if val, ok := ctx.Value(CtxKeyUser).(string); ok {
        fmt.Printf("value is string: %v %t\n", val, ok) // value is string: value true
    }

    // Проверим наличие в базовом контексте
    val := ctxBase.Value(CtxKeyUser)
    fmt.Println("from base ctx:", val)
}
// ❯ go run .
// value is string: value true
// from base ctx: <nil>
```

ВАЖНО!!!

Для ключей лучше использовать свой тип.

Обычно context.WithValue нужен для передачи по цепочки не чувствительных параметров, но нужных для дальнейшего использования:

- request_id
- user_id
- trace_id
- auth claims
- etc.

context.WithCancel

`context.WithCancel(parent context.Context) (ctx context.Context, cancel context.CancelFunc)` - создает контекст, который можно вручную отменить через cancel().

Пример:

```
ctx, cancel := context.WithCancel(context.Background())
defer cancel()
```

`cancel()` закрывает канал ctx.Done().

Пример:

```
package main

import (
    "context"
    "fmt"
)

func main() {
    ctx, cancel := context.WithCancel(context.Background())

    cancel()

    select {
    case <-ctx.Done():
        fmt.Println(ctx.Err())
    default:
        fmt.Println("not canceled")
    }
```

```
}
// context canceled
```

Используют, когда нужно вручную остановить выполнение:

```
package main

import (
    "context"
    "fmt"
    "time"
)

func worker(ctx context.Context) {
    i := 0
    for {
        fmt.Println(i)
        i++
        time.Sleep(time.Second)
    }
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())

    go worker(ctx)

    time.Sleep(3 * time.Second)
    cancel()
}
// ❯ go run .
// 0
// 1
// 2
```

Нужно еще понимать, что если мы условно сделаем select и будем ждать `ctx.Done()` и контекст main горутины отменится, то worker может не успеть словить ctx.Done() и просто тоже отменится:

```
package main

import (
    "context"
    "fmt"
    "time"
)

func worker(ctx context.Context) {
    i := 0
```

```
    for {
        select {
        case <-ctx.Done():
            fmt.Println("worker done, ctx canceled")
            return
        default:
            fmt.Println(i)
            i++
            time.Sleep(time.Second)
        }
    }
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())

    go worker(ctx)

    time.Sleep(3 * time.Second)
    cancel()
}
// ❯ go run .
// 0
// 1
// 2
```

Чтобы подождать, нужно привычно использовать wg:

```
package main

import (
    "context"
    "fmt"
    "sync"
    "time"
)

func worker(ctx context.Context, wg *sync.WaitGroup) {
    defer wg.Done()
    i := 0
    for {
        select {
        case <-ctx.Done():
            fmt.Println("worker done, ctx canceled")
            return
        default:
            fmt.Println(i)
            i++
            time.Sleep(time.Second)
        }
    }
```

```
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())

    wg := sync.WaitGroup{}
    wg.Add(1)

    go worker(ctx, &wg)

    time.Sleep(3 * time.Second)
    cancel()

    wg.Wait()
}
// ❯ go run .
// 0
// 1
// 2
// worker done, ctx canceled
```

context.WithTimeout

`context.WithTimeout(parent context.Context, timeout time.Duration) (ctx context.Context, cancel context.CancelFunc)` - создает контекст, который автоматически отменяется через timeout.

Пример:

```
ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()
```

Этот контекст отменится через 2 секунды после создания сам.

После timeout:

```
ctx.Err() // context deadline exceeded
```

Пример:

```
package main

import (
    "context"
    "fmt"
    "time"
)
```

```
func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    ticker := time.NewTicker(time.Second)

    for {
        select {
        case <-ticker.C:
            fmt.Println("tick")

        case <-ctx.Done():
            fmt.Printf("ctx done: %v\n", ctx.Err())
            return // это обяз, иначе из for не выйдем
        }
    }
}
// ❯ go run .
// tick
// tick
// tick
// ctx done: context deadline exceeded
```

Если хотим без return, то правильнее вынести в другую функцию (запустить не в горутине):

```
package main

import (
    "context"
    "fmt"
    "time"
)

func tickTack(ctx context.Context) error {
    ticker := time.NewTicker(time.Second)
    defer ticker.Stop()

    for {
        select {
        case <-ticker.C:
            fmt.Println("tick")
        case <-ctx.Done():
            fmt.Printf("ctx done: %v\n", ctx.Err())
            return ctx.Err()
        }
    }
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
```

```
    defer cancel()

    ticker := time.NewTicker(time.Second)
    defer ticker.Stop()

    err := tickTack(ctx)
    fmt.Println("main go err: ", err)
}
// ❯ go run .
// tick
// tick
// tick
// ctx done: context deadline exceeded
// main go err:  context deadline exceeded
```

context.WithDeadline

`context.WithDeadline(parent context.Context, deadline time.Time) (ctx context.Context, cancel context.CancelFunc)` - создает контекст, который автоматически отменяется в определенный момент времени.

Пример:

```
deadline := time.Now().Add(5 * time.Second)
ctx, cancel := context.WithDeadline(context.Background(), deadline)
defer cancel()
```

В принципе работает как с timeout, только отмена в конкретный момент времени.

context.WithCancelCause

`context.WithCancelCause(parent context.Context) (ctx context.Context, cancel context.CancelCauseFunc)` - создает контекст, который можно вручную отменить с конкретной причиной.

Обычный `ctx.Err()` возвращает только стандартные ошибки:

- context.Canceled
- context.DeadlineExceeded

А `context.Cause(ctx)` позволяет получить конкретную причину отмены:

```
ctx, cancel := context.WithCancelCause(context.Background())
```

```
cancel(errors.New("user deleted request"))

<-ctx.Done()

fmt.Println(ctx.Err())          // context canceled
fmt.Println(context.Cause(ctx)) // user deleted request
```

Используется, когда нужно точно знать, почему контекст отменен.

`context.Cause(ctx)` возвращает то же значение, что и `ctx.Err()`.

context.AfterFunc

`context.AfterFunc(ctx context.Context, f func()) (stop func() bool)` - регистрирует функцию, которая будет вызвана после отмены контекста.

Пример:

```
package main

import (
    "context"
    "fmt"
    "time"
)

func tickTack(ctx context.Context) error {
    ticker := time.NewTicker(time.Second)
    defer ticker.Stop()

    for {
        select {
        case <-ticker.C:
            fmt.Println("tick")
        case <-ctx.Done():
            fmt.Printf("ctx done: %v\n", ctx.Err())
            return ctx.Err()
        }
    }
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    stop := context.AfterFunc(ctx, func() {
        fmt.Println("context canceled, after func executed")
    })
    defer stop()
    defer cancel()
```

```
    ticker := time.NewTicker(time.Second)
    defer ticker.Stop()

    err := tickTack(ctx)
    fmt.Println("main go err: ", err)
}
// ❯ go run .
// tick
// tick
// tick
// context canceled, after func executed
// ctx done: context deadline exceeded
// main go err:  context deadline exceeded
```

Context SHPORA

`context.Background()`

- Создать пустой корневой контекст.

`context.TODO()`

- Временный пустой контекст, когда пока непонятно, какой передать.

`context.WithCancel(parent)`

- Создать дочерний контекст с ручной отменой.

`context.WithTimeout(parent, timeout)`

- Создать дочерний контекст с timeout.

`context.WithDeadline(parent, deadline)`

- Создать дочерний контекст с deadline.

`context.WithValue(parent, key, val)`

- Создать дочерний контекст со значением.

`ctx.Done()`

- Канал, который закрывается при отмене контекста.

`ctx.Err()`

- Причина отмены: context.Canceled или context.DeadlineExceeded.

`ctx.Deadline()`

- Получить deadline, если он есть.

`ctx.Value(key)`

- Получить значение по ключу.

`context.WithCancelCause(parent)`

- Создать context с возможностью указать причину отмены.

`context.Cause(ctx)`

- Получить конкретную причину отмены.

Основные функции:

```
context.Background() context.Context
context.WithCancel(parent context.Context) (ctx context.Context, cancel context.CancelFunc)
context.WithDeadline(parent context.Context, deadline time.Time) (ctx context.Context, cancel
context.WithTimeout(parent context.Context, timeout time.Duration) (ctx context.Context, cance
context.WithValue(parent context.Context, key, val any) context.Context
```

Garbage collector

`GC` - Garbage collector, сборщик мусора.

Он автоматически освобождает память кучи, которую уже не использует программа.

Куча и стэк

Для даунов типа меня, которые это не помнят с курса архитектур вычислительных систем:

Есть условное разделение памяти на секцию кода, кучу и стэк.

В плоской подели памяти это выглядит примерно так:

```
низкие адреса памяти                          высокие адреса памяти
+------------------+----------------------+--------------------+
| code/data segment | heap grows ->        | <- stack grows     |
+------------------+----------------------+--------------------+
```

Это норм модель, если пытаться понять assembler, но в рамках современных OS и Go лучше воспринимать так:

- stack - отдельная область памяти для выполнения конкретной горутины
- heap - общая область памяти для объектов, которые могут жить дольше текущего вызова функции

Потому что Go runtime сам управляет кучей и у каждой горутины есть свой маленький growable stack.

Если коротко про основную разницу:

Stack

- быстрый
- принадлежит конкретной goroutine
- используется для вызовов функций
- освобождается автоматически при выходе из функции
- не требует работы GC для освобождения каждого объекта

Heap

- общий для программы
- используется для объектов, которые могут жить дольше функции, точнее, время жизни которых нельзя ограничить текущим stack frame
- освобождается garbage collector'ом
- дороже, чем stack
- создает нагрузку на GC

Стэк

Стэк нужен для выполнения функций. Когда вызывается функция, создается stack frame - кусок стека для текущего вызова.

В нем могут лежать:

- локальные переменные
- аргументы функции
- возвращаемые значения
- часть временных данных
- служебная информация для возврата из функции

В Go у каждой горутины свой стек, то есть упрощенно:

```
goroutine 1 -> stack 1
goroutine 2 -> stack 2
```

```
goroutine 3 -> stack 3
```

Стек горутины изначально может быть маленький, то он growable и по надобности может расти.

Куча

Куча нужна для объектов, которые нельзя безопасно хранить только в стэке текущей функции.

Например:

```
func NewUser() *User {
    u := User{Name: "Vlad"}
    return &u
}
```

Тут `u` создан внутри функции, но мы возвращаем указатель на него. Если бы он лежал на стеке, то при выходе из функции он был удален и снаружи по указателю мы бы попали в невалидную память.

Поэтому компилятор понимает, что `u` должен жить дольше, чем вызов функции и кладет `u` в heap.

Тут если точнее, может ли компилятор безопасно разместить объект на стеке? Если нет, то heap.

escape analysis

В Go решение о размещении объектов на стеке и в heap принимает компилятор через escape analysis.

Escape analysis отвечает на вопрос:

- убегает ли значение за пределы текущей функции?

Если не убегает: можно положить на stack

Если убегает: скорее всего heap

Но тут проблема, что по синтаксису Go нельзя однозначно определить, попадет ли значение на stack или heap.

Единственное, основываясь на чем можно предположить, это по передаче указателей в функцию или создании указателей на объекты внутри функции.

Механизм работы

Основные задачи GC:

- автоматически находить объекты в heap, которые больше не достижимы из программы
- освобождать память, занятую этими объектами

Что чистит GC:

В основном чистит heap, потому что стековые данные обычно освобождаются проще: функция завершилась - ее stack frame больше не нужен.

Но при этом GC должен сканировать стеки горутин, потому что на стеках могут лежать ссылки на heap объекты.

Основной принцип работы GC:

Roots - это начальные точки, откуда GC начинает искать живые объекты.

К roots относятся:

- стеки горутин
- глобальные переменные
- регистры
- runtime-структуры

GC не понимает, нужен ли объект логически, он может узнать только `достижим ли объект по ссылкам из roots`.

Если достижим, то объект нужен

Если не достижим, то объект - мусор

STW

STW - момент, когда Go runtime ненадолго останавливает выполнение всех gouroutine.

Это нужно, чтобы GC безопасно переключил режим работы и завершил фазу.

В Go GC старается делать STW-паузы короткими.

Алгоритм действий:

Используется подход: *concurrent tri-color mark-and-sweep*

Mark (маркировка):

Для маркировки используется tri-color marking:

- white - объект еще не обработан, потенциальный мусор
- gray - объект живой, но его ссылки еще не просмотрены (еще до конца не обработали)

- black - объект живой, его ссылки уже просмотрены

Алгоритм маркировки такой:

1. В начале все объекты white.
2. Roots становятся gray.
3. GC берет gray-объект.
4. Сканирует его ссылки.
5. Найденные white-объекты становятся gray.
6. Сам объект становится black.
7. Когда gray-объектов больше нет, все оставшиеся white — мусор.

Все это GC выполняет параллельно с работой программы. Но так как полностью без остановок нельзя, есть STW (Stop The World).

Примерный цикл GC упращенно такой:

1. Sweep termination - короткий STW
2. Mark phase - concurrent marking
3. Mark termination - короткий STW
4. Sweep phase - concurrent/background sweeping

Sweep termination

Перед новым циклом GC нужно завершить предыдущий sweep. Для этого runtime выполняет короткий STW.

Если упращенно:

- остановить мир
- убедиться, что предыдущая отчистка завершена
- включить write barrier
- начать mark-фазу
- запустить мир обратно

Mark phase

Основная фаза GC.

GC начинает от roots и помечает все достижимые heap-объекты. Эта фаза работает в основном параллельно с работой программы, то есть во время этой фазы программа продолжает менять ссылки, создавать объекты, работать с heap.

Из-за этого нужен специальный механизм - write barrier.

Write barrier - это дополнтельный runtime-механизм при записи ссылок, который помогает GC не потерять объект.

Это нужно так как GC может уже пометить объект, как черный, а затем положить в него ссылку на другой объект, который GC еще не видел. Тогда GC ошибочно может подумать, что другой объект не достижим.

Mark termination

Когда mark-фаза закончена, runtime делает еще короткий STW.

На этой фазе:

- останавливает мир
- доделывает оставшуюся mark-работу
- отключает write barrier
- подготавливает sweep
- запускает мир обратно

После этой фазы GC уже знает, какие объекты живые, а какие мусор.

Sweep phase

Sweep освобождает память от объектов, которые не были помечены как живые.

Эта фаза может выполняться concurrently/background.

Упрощенно:

- GC проходит по heap spans
- освобождает непомеченные объекты
- возвращает память в allocator

Важный момент, что освободить объект - не всегда значит сразу вернуть память ОС. Часто runtime оставляет память себе (внутреннему аллокатору), чтобы потом быстрее переиспользовать ее под новые объекты.

Запуск GC

GC не запускается каждые N секунд.

Основной триггер к запуску GC - рост кучи.

Это регулируется через `live heap`, `GOGC`, `memory limit`.

`live heap` - это сколько в MB осталось живых объектов.

`GOGC` - это основной параметр частоты GC, по умолчанию `GOGC=100`. Это значит, что следующий GC начнется, когда heap вырастет примерно на 100% от live heap после прошлого GC.

Например, после прошлого GC:

live heap = 100 MB

GOGC = 100

То следующий GC будет, когда heap вырастет примерно до 200 MB

Задать GOGC

Через env:

```
GOGC=50 go run .
```

Или отключить полностью:

```
GOGC=off go run .
```

Если хотим в коде, то через пакет `runtime/debug`:

```
import "runtime/debug"
debug.SetGCPercent(50)
```

`debug.SetGCPercent` меняет процент GC и возвращает предыдущее значение:

```
package main

import (
    "fmt"
    "runtime"
    "runtime/debug"
)

type User struct {
    Name string
}

func main() {
    u := &User{Name: "Vlad"}

    fmt.Printf("%+v\n", u)
```

```
    u = nil

    runtime.GC()

    fmt.Println(debug.SetGCPercent(50))
}

// ❯ go run .
// &{Name:Vlad}
// 100
// ❯ GOGC=off go run .
// &{Name:Vlad}
// -1
```

Если хочется руками в коде запустить GC, то можно использовать `runtime.GC()`.

GOMEMLIMIT

`GOMEMLIMIT` - мягкий memory limit для Go runtime.

Он говорит Go runtime, в рамках какого лимита нужно держать использование памяти.

Пример:

```
GOMEMLIMIT=512MB go run .
```

Или в коде:

```
debug.SetMemoryLimit(512 << 20)
```

Важный момент, что `GOMEMLIMIT` не жесткий запрет на память, а ограничение на runtime, из-за которого GC может запускаться чаще.

Для того, чтобы посмотреть работу GC можно использовать:

```
GODEBUG=gctrace=1 go run .
```

```
package main

import (
    "fmt"
    "runtime"
    "sync"
```

```
    localcache "github.com/vladlim/go-tour/local_cache"
)

func main() {
    localCache := localcache.New[string, int]()

    localCache.Set("Vlad", 10)
    localCache.Set("Alex", -10)

    var wg sync.WaitGroup
    wg.Add(10)

    for i := range 10 {
        go process(i, localCache, &wg)
    }

    wg.Wait()

    localCache = nil

    fmt.Println("SET PTR NIL")

    runtime.GC()

    lc2 := localcache.New[int, int]()
    lc2.Set(10, 10)
    lc2.Set(12, -10)

    runtime.GC()
}
// ❯ GODEBUG=gctrace=1 go run .
// gc 1 @0.048s 0%: 0.016+1.0+0.023 ms clock, 0.033+0.28/0.54/0+0.046 ms cpu, 3->4->1 MB, 4 MB
// gc 2 @0.058s 2%: 0.044+2.0+0.012 ms clock, 0.089+1.6/0.14/0+0.024 ms cpu, 3->4->1 MB, 4 MB
// gc 3 @0.062s 3%: 0.025+1.5+0.017 ms clock, 0.050+1.3/0.24/0+0.035 ms cpu, 3->4->1 MB, 4 MB
// gc 4 @0.064s 5%: 0.18+4.3+0.005 ms clock, 0.37+2.1/0.49/0+0.010 ms cpu, 3->3->1 MB, 4 MB go
// gc 5 @0.070s 6%: 0.39+1.5+0.003 ms clock, 0.79+0.74/0.61/0+0.007 ms cpu, 3->4->1 MB, 4 MB g
// gc 6 @0.073s 7%: 0.019+1.7+0.020 ms clock, 0.038+0.16/0.89/0.76+0.040 ms cpu, 3->4->1 MB, 4
// gc 7 @0.077s 7%: 0.019+0.85+0.008 ms clock, 0.039+0.44/0.038/0.75+0.017 ms cpu, 3->4->1 MB,
// gc 8 @0.078s 7%: 0.016+0.91+0.010 ms clock, 0.033+0.88/0.046/0+0.021 ms cpu, 3->4->1 MB, 4
// gc 9 @0.080s 7%: 0.031+1.7+0.007 ms clock, 0.063+0.39/0.27/0.95+0.015 ms cpu, 3->4->2 MB, 4
// gc 10 @0.083s 8%: 0.017+0.82+0.002 ms clock, 0.034+0.41/0.17/0.42+0.004 ms cpu, 4->5->2 MB,
// # github.com/vladlim/go-tour
// ./main.go:32:20: localCache.New undefined (type *localcache.LocalCache[string, int] has no
// ❯ GODEBUG=gctrace=1 go run .
// gc 1 @0.041s 0%: 0.051+0.96+0.045 ms clock, 0.10+0.30/0.38/0+0.091 ms cpu, 3->4->1 MB, 4 MB
// gc 2 @0.047s 2%: 0.047+1.3+0.009 ms clock, 0.095+1.1/0.087/0+0.019 ms cpu, 3->4->2 MB, 4 MB
// gc 3 @0.050s 3%: 0.034+2.3+0.005 ms clock, 0.069+0.98/0.050/0+0.011 ms cpu, 3->4->1 MB, 4 M
// gc 4 @0.053s 4%: 0.12+1.0+0.002 ms clock, 0.24+1.2/0.061/0+0.005 ms cpu, 3->4->1 MB, 4 MB g
// gc 5 @0.055s 5%: 0.15+4.0+0.010 ms clock, 0.31+0.56/0.96/0+0.020 ms cpu, 3->4->2 MB, 4 MB g
// gc 6 @0.063s 5%: 0.045+1.0+0.003 ms clock, 0.091+0.21/0.33/0.69+0.006 ms cpu, 4->4->1 MB, 4
// gc 7 @0.065s 6%: 0.018+2.3+0.017 ms clock, 0.036+0.36/0.85/1.2+0.035 ms cpu, 3->4->1 MB, 4
// gc 8 @0.068s 6%: 0.038+2.8+0.006 ms clock, 0.076+0.40/0.54/0+0.012 ms cpu, 3->5->3 MB, 4 MB
// gc 9 @0.073s 6%: 0.042+1.1+0.008 ms clock, 0.084+0.41/0.29/0.71+0.016 ms cpu, 5->6->2 MB, 6
```

```
// # github.com/vladlim/go-tour
// gc 1 @0.002s 4%: 0.014+0.79+0.065 ms clock, 0.028+0.080/0.088/0.54+0.13 ms cpu, 3->4->3 MB,
// gc 2 @0.006s 3%: 0.012+0.94+0.029 ms clock, 0.025+0.065/0.16/0.70+0.058 ms cpu, 6->7->6 MB,
// gc 3 @0.020s 3%: 0.020+1.2+0.11 ms clock, 0.040+0.28/0.31/0.48+0.22 ms cpu, 11->12->9 MB, 1
// i=9 key=Alex val=-10
// i=9 key=Alex val=-11
// i=0 key=Vlad val=10
// i=0 key=Vlad val=11
// i=1 key=Alex val=-11
// i=1 key=Alex val=-12
// i=2 key=Vlad val=11
// i=2 key=Vlad val=12
// i=3 key=Alex val=-12
// i=3 key=Alex val=-13
// i=4 key=Vlad val=12
// i=4 key=Vlad val=13
// i=5 key=Alex val=-13
// i=5 key=Alex val=-14
// i=6 key=Vlad val=13
// i=6 key=Vlad val=14
// i=7 key=Alex val=-14
// i=7 key=Alex val=-15
// i=8 key=Vlad val=14
// i=8 key=Vlad val=15
// SET PTR NIL
// gc 1 @0.000s 5%: 0.012+0.11+0.004 ms clock, 0.025+0/0.045/0.076+0.008 ms cpu, 0->0->0 MB, 4
// gc 2 @0.000s 14%: 0.15+0.12+0.004 ms clock, 0.30+0/0.070/0.044+0.008 ms cpu, 0->0->0 MB, 4
```

Также можно использовать `runtime.MemStats` для получения статичтики по аллоцированию и чистке памяти:

```
package main

import (
    "fmt"
    "runtime"
    "time"
)

func printMem() {
    var m runtime.MemStats
    runtime.ReadMemStats(&m)

    fmt.Printf("Alloc=%d KB TotalAlloc=%d KB NumGC=%d\n",
        m.Alloc/1024,
        m.TotalAlloc/1024,
        m.NumGC,
    )
}

func main() {
```

```
    printMem()

    for i := 0; i < 10; i++ {
        _ = make([]byte, 10*1024*1024)
        printMem()
        time.Sleep(200 * time.Millisecond)
    }

    runtime.GC()
    printMem()
}
// ❯ GODEBUG=gctrace=1 go run .
// gc 1 @0.046s 1%: 0.12+0.84+0.039 ms clock, 0.25+0.39/0.33/0+0.078 ms cpu, 3->4->1 MB, 4 MB
// gc 2 @0.053s 1%: 0.020+1.0+0.003 ms clock, 0.040+0.99/0.047/0+0.007 ms cpu, 3->4->2 MB, 4 M
// gc 3 @0.056s 2%: 0.025+1.1+0.003 ms clock, 0.051+0.49/0.50/0.070+0.007 ms cpu, 3->5->2 MB,
// gc 4 @0.058s 3%: 0.024+1.1+0.001 ms clock, 0.048+1.0/0.078/0+0.003 ms cpu, 4->4->1 MB, 4 MB
// gc 5 @0.060s 4%: 0.11+2.0+0.003 ms clock, 0.22+0.45/1.3/0.046+0.006 ms cpu, 3->4->1 MB, 4 M
// gc 6 @0.064s 5%: 0.030+1.0+0.009 ms clock, 0.060+0.55/0.081/0.83+0.018 ms cpu, 3->4->1 MB,
// gc 7 @0.067s 6%: 0.12+1.3+0.063 ms clock, 0.25+0.68/0.61/0+0.12 ms cpu, 3->4->1 MB, 4 MB go
// gc 8 @0.069s 6%: 0.021+0.83+0.032 ms clock, 0.042+0.44/0.081/0.48+0.065 ms cpu, 3->4->1 MB,
// gc 9 @0.071s 7%: 0.013+3.4+0.005 ms clock, 0.027+1.7/0.65/0+0.010 ms cpu, 3->4->2 MB, 4 MB
// gc 10 @0.076s 7%: 0.022+1.0+0.008 ms clock, 0.045+0.074/0.092/0.74+0.017 ms cpu, 4->4->1 MB
// # github.com/vladlim/go-tour
// gc 1 @0.002s 8%: 0.055+1.6+0.008 ms clock, 0.11+0.66/0/0+0.017 ms cpu, 3->5->5 MB, 4 MB goa
// gc 2 @0.007s 9%: 0.018+2.3+0.034 ms clock, 0.037+0.11/0.98/0.95+0.069 ms cpu, 9->9->8 MB, 1
// gc 3 @0.034s 3%: 0.13+1.4+0.036 ms clock, 0.26+0/0.31/1.0+0.073 ms cpu, 15->15->9 MB, 18 MB
// Alloc=52 KB TotalAlloc=52 KB NumGC=0
// Alloc=10295 KB TotalAlloc=10295 KB NumGC=0
// gc 1 @0.000s 7%: 0.008+0.16+0.014 ms clock, 0.017+0.045/0.031/0+0.028 ms cpu, 10->10->0 MB,
// gc 2 @0.201s 0%: 0.018+0.33+0.16 ms clock, 0.036+0/0.076/0.066+0.33 ms cpu, 10->10->10 MB,
// Alloc=10291 KB TotalAlloc=20535 KB NumGC=2
// Alloc=20532 KB TotalAlloc=30776 KB NumGC=2
// gc 3 @0.406s 0%: 0.13+0.27+0.007 ms clock, 0.26+0.066/0.035/0+0.015 ms cpu, 20->20->0 MB, 2
// gc 4 @0.607s 0%: 0.12+0.25+0.10 ms clock, 0.25+0/0.13/0+0.20 ms cpu, 10->10->10 MB, 10 MB g
// Alloc=10291 KB TotalAlloc=41016 KB NumGC=4
// gc 5 @0.808s 0%: 0.024+0.55+0.12 ms clock, 0.049+0/0.10/0.055+0.24 ms cpu, 20->20->10 MB, 2
// Alloc=10292 KB TotalAlloc=51257 KB NumGC=5
// gc 6 @1.013s 0%: 0.11+0.34+0.030 ms clock, 0.22+0/0.19/0+0.060 ms cpu, 20->20->10 MB, 20 MB
// Alloc=10292 KB TotalAlloc=61497 KB NumGC=6
// gc 7 @1.214s 0%: 0.018+0.16+0.034 ms clock, 0.036+0/0.081/0.049+0.069 ms cpu, 20->20->10 MB
// Alloc=10292 KB TotalAlloc=71737 KB NumGC=7
// gc 8 @1.415s 0%: 0.016+0.16+0.028 ms clock, 0.033+0/0.072/0.053+0.057 ms cpu, 20->20->10 MB
// Alloc=10292 KB TotalAlloc=81978 KB NumGC=8
// gc 9 @1.616s 0%: 0.099+0.30+0.13 ms clock, 0.19+0/0.15/0+0.26 ms cpu, 20->20->10 MB, 20 MB
// Alloc=10292 KB TotalAlloc=92218 KB NumGC=9
// gc 10 @1.817s 0%: 0.030+0.23+0.030 ms clock, 0.061+0/0.12/0.071+0.061 ms cpu, 20->20->10 MB
// Alloc=10292 KB TotalAlloc=102459 KB NumGC=10
// gc 11 @2.019s 0%: 0.024+0.071+0.005 ms clock, 0.048+0/0.060/0.046+0.011 ms cpu, 10->10->0 M
// Alloc=52 KB TotalAlloc=102459 KB NumGC=11
```

Памятка по GC

Что лучше помнить:

- Чем больше heap allocations, тем больше работы для GC (а это может создать нагрузку на CPU)
- Долгие ссылки = объект не освобождается
- Большие slices/map могут держать память
- Много goroutine влекут много стеков для сканирования GC
- GC не удаляет то, на что еще есть ссылка

Частые причины давления на GC:

- частый `fmt.Sprintf()` в горячих циклах (из-за создания новых строк)
- постоянные `json.Marshal()` и `json.Unmarshal()`
- создание больших временных slices/maps
- append без capacity
- удержание больших underlying arrays маленькими slices
- кэши
- глобальные map/slices, которые только растут
- много указателей в структурах

Эта часть была взята из гпт, лучше конечно почитать документацию и посмотреть лекции от создателей Go

Scheduler

`Scheduler` - это часть Go runtime, которая занимается планированием выполнения горутин.

Он решает:

- какая goroutine сейчас будет выполняться
- на каком OS thread она будет выполняться
- когда переключиться на другую goroutine

Горутин можно создать намного больше чем OS threads и CPU ядер, значит кто-то должен решать:

- какие выполнять сейчас
- какие поставить в очередь
- какие усыпить
- какие разбудить

Этим занимается Scheduler.

То есть scheduler "раскладывает" много горутин на меньшее количество OS threads.

GMP

Обычно scheduler объясняют через модель GMP (goroutine-machine-processor).

Goroutine

`G` - goroutine

когда мы пишем `go SomeFunc()`, создается новая горутина, у которой есть:

- свой стек
- состояние
- информация о функции
- ссылка на место выполнения
- служебная информация runtime

Machine

`M` - OS thread

То есть настоящий поток операционной системы.

Go runtime просит ОС создать/использует OS threads внутри процесса и выполняет на них горутины.

Processor

`P` - логический процессор Go runtime

У него есть:

- локальная очередь runnable goroutines
- часть runtime-ресурсов
- кэш allocator'a
- scheduler state

Количество `P` определяется `GOMAXPROCS`.

P.S. Вообще тут как-то непонятно доконца.

`P` - внутренняя сущность Go runtime (это не железо), которая дает OS thread право выполнять Go-код. Это не поток и не ядро, это runtime-слот, без которого M не может выполнять Go-код.

Связь GMP

Чтобы выполнить Go-код, нужна связка M+P+G

То есть:

- OS thread M берет processor P
- из очереди P берет goroutine G
- выполняет G

Почему нельзя просто goroutine -> thread?

Потому что P нужен как промежуточная сущность, которая хранит ресурсы для выполнения Go кода.

Например, если M ушел в blocking syscall, то он может отдать свой P другому M, так остальные goroutine продолжат выполняться.

GOMAXPROCS

`GOMAXPROCS` - это количество `P`, то есть максимальное количество OS threads, которые одновременно могут выполнять Go-код.

Если установить во время выполнения программы:

```
runtime.GOMAXPROCS(4)
```

Это будет значить, что одновременно Go-код могут выполнять максимум 4 `M`, привязанные к 4 `P`.

Но горутин может быть гораздо больше:

```
goroutine = 10000
GOMAXPROCS = 4
```

Тогда одновременно на CPU будут выполняться только примерно 4 горутины, остальные будут ждать своей очереди или будут заблокированы.

Тут есть замечание, что привязка к CPU cores не строгая. ОС сама решает, на каких ядрах выполнять эти OS threads. GOMAXPROCS это про ограничение количества на единовременное выполнение.

Состояния goroutine

Горутина может быть в разных состояниях.

Основные:

- running — сейчас выполняется
- runnable — готова выполняться, ждет очереди
- waiting — ждет channel/mutex/timer/network I/O
- syscall — выполняет системный вызов
- dead — завершилась

Пример `running`:

```
go func() {
    for i := 0; i < 10; i++ {
        fmt.Println(i)
    }
} ()
```

Когда scheduler дал этой горутине CPU, она в состоянии `running`

Пример `waiting`:

```
ch := make(chan int)

go func() {
    v := <-ch
    fmt.Println(v)
}()
```

Пока в `ch` ничего нет, горутина ждет.

Она не крутится в цикле и не ест CPU.

В этот момент она в состоянии `waiting`.

Scheduler в это время выполняет другие goroutine.

Пример `runnable`, это если горутина готова работать, но сейчас все P заняты другими горутинами.

Очереди Scheduler

У scheduler есть несколько очередей

Основные:

- local run queue у каждого P
- global run queue

Новые горутины обычно попадают в local run queue текущего P.

Local run queue

У каждого `P` есть своя локальная очередь runnable goroutines.

Например:

- P1 queue: G1, G2, G3
- P2 queue: G4, G5
- P3 queue: G6

Когда `P` ищет работу, он сначала смотрит на свою локальную очередь.

Это ускоряет работу, потому что меньше синхронизации между потоками.

Global run queue

Есть также глобальная очередь runnable goroutines.

Она нужна для балансировки и некоторых runtime-сценариев.

Scheduler периодически проверяет global queue, чтобы goroutine оттуда не голодали.

Work stealing

Если у одного `P` закончились goroutine, то он может украсть часть работы другого `P`

Например:

- P1 queue: G1, G2, G3, G4, G5, G6
- P2 queue: empty

`P2` может украсть часть горутин у `P1`

- P1 queue: G1, G2, G3
- P2 queue: G4, G5, G6

Это нужно, чтобы CPU не простаивали.

Что происходит при `go f()`

Когда мы пишем:

```
go f()
```

Происходит примерно следующее:

1. runtime создает новую goroutine G
2. выделяет ей начальный стек
3. записывает, какую функцию надо выполнить
4. кладет G в очередь runnable goroutines
5. scheduler позже берет эту G и запускает

То есть не факт, что `f` начнет в этот же момент выполняться.

Это означает, что мы создали горутину и поставили ее на выполнение. А начнется она когда освободится `P` и возьмет ее в работу (не обязательно этот P, может быть work stealing, поэтому корректнее так: когда scheduler выберет эту G для выполнения на каком-то M, связанном с P.)

По этой причине `main` не ждет другие горутины. Он завершается, процесс завершается, все горутины умирают. Scheduler не обязан "довыполнить" все горутины перед завершением программы. Для этого есть `wg` .

Поведение при блокировках

При блокировке на channel

```
func worker(ch <-chan int) {
    v := <-ch
```

```
    fmt.Println(v)
}
```

Если канал пустой:

- G переходит в waiting
- она привязывается к ожиданию этого channel
- M с привязанным P освобождаются для выполнения другой runnable G

Когда кто-то отправит значение:

```
ch <- 10
```

runtime разбудит waiting горутину и переведет ее в состояние runnable.

Потом scheduler снова поставит ее на выполнение.

При time.Sleep

```
time.Sleep(time.Second)
```

Горутина не занимают CPU весь период сна.

Она становится waiting на timer.

Когда timer сработает, runtime сделает горутину runnable.

При mutex

```
mu.Lock()
```

Если mutex свободен — goroutine продолжает работу.

Если mutex занят — goroutine блокируется, становится waiting.

Когда mutex освобождается, одна из ожидающих goroutine становится runnable.

Network I/O и netpoller

Go хорошо работает с большим числом сетевых соединений из-за netpoller.

Например:

```
conn.Read(buf)
```

Если данных пока нет, goroutine не должна держать OS thread заблокированным.

Runtime регистрирует ожидание в netpoller и паркует goroutine.

Когда socket готов к чтению, netpoller сообщает runtime:

эта goroutine теперь runnable

И scheduler ее запускает.

Упрощенно:

- goroutine ждет network I/O
- runtime паркует goroutine
- OS thread свободен для другой работы
- socket стал ready
- netpoller будит goroutine

При syscall

Если горутина делает blocking syscall, то OS thread может реально заблокироваться.

В таком сценарии чтобы другие горутины не стояли, runtime может отцепить `P1` от заблокированного `M1` и дать его другому thread `M2`

Когда syscall закончится, `M1` вернется и попробоует снова получить `P` .

Если свободного `P` нет, горутина станет runnable и будет ждать.

Это важно, так как без такой модели один blocking syscall мог бы заблокировать выполнение других горутин на этом `P`

С моделью G/M/P runtime сохраняет параллелизм.

Preemption

`Preemption` - это принудительное прерывание долго работающей горутины.

Чтобы горутина с тяжелыми вычислениями, которая долго не блокируется и не вызывает функций не могла слишком долго удерживать поток, Go runtime прерывает ее и дает

выполняться другим горутинам.

Поэтому основной совет:

Не писать бесконечные CPU loops без условий выхода.

Общая схема и заключение

Очень верхеуровнево, поскольку до конца в голове не смог правильно все уложить.

Есть процессор, у него есть физические ядра (допустим обычно 16 или 8).

Есть системные потоки OS threads, их может быть условно 3000.

Есть планировщик OS, который занимается тем же самым, что планировщик Go, но с системными потоками и ядрами CPU. То есть на каком ядре какой поток сейчас запустить, с какого снять, перекинуть на другое ядро, какой приостановить и т.д.

В один момент времени реально может выполняться столько потоков, сколько ядер CPU (если нет дополнительных механизмов с логическими ядрами, что на одном ядре может выполняться > 1 OS thread в один момент).

Есть планировщик Go, который раскидывает горутины по OS threads.

Есть горутины, которые выполняются на каком-либо OS thread.

То есть слои такие:

```
Go goroutines
      |
Go scheduler
      |
OS threads
      |
OS scheduler
      |
CPU cores
```

В модели GMP:

- G - горутина
- M - OS thread
- P - логический процессор Go runtime

Схема такая:

```
Goroutines:
  G1 G2 G3 G4 G5 ...

Go Scheduler:
  распределяет G по P/M

OS Threads:
  M1 M2 M3 M4 ...

OS Scheduler:
  распределяет M по CPU cores

CPU cores:
  C1 C2 C3 C4 ...
```

То есть имеется два уровня планирования:

- OS scheduler планирует OS threads на CPU cores
- Go scheduler планирует goroutines на OS threads

Чтобы горутина выполнялась, нужен M с привязанным к ней P. То есть нужен OS thread, к которому привязан логический процессор Go runtime, у которого есть:

- local run queue goroutine
- runtime resources
- часть allocator/cache
- scheduler state

Scheduler отчасти связан с работой GC, поскольку GC использует механизмы runtime/scheduler чтобы:

- останавливать goroutine на safepoints
- запускать GC worker goroutines
- координировать STW-паузы
- сканировать стеки goroutine

Scheduler/runtime знает о блокировках, которые происходят через механизмы Go/runtime:

- channel
- select
- time.Sleep / timers
- mutex
- network I/O через netpoller
- syscall
- GC safepoints

При получении сигналов о блокировках/разблокировках, сне и т.д. scheduler кладет их в очередь. Потом любой M с P может взять ее на выполнение.

Вообще тема достаточно сложная, поэтому представлять все это в жесткой схеме "ядро->поток->P->горутина" не нужно, так как в реальности это динамическая система очередей и привязок.

Строки

Строка в Go - неизменяемая последовательность байтов (можно воспринимать как срез байт только для чтения).

`string = []byte = []uint8`

В Go строки имеют кодировку UTF-8.

`UTF-8 = Unicode Transformation Format 8`

1 byte = число от 0 до 255

В `UTF-8` :

Символ	Количество байтов
ASCII(Латиница)	1 byte
Кириллица	2 byte
Часть символов	3 byte
Emoji	4 byte

Строки *immutable*, то есть их нельзя изменить по индексу:

```
s := "Hello"
s[0] = 'J' // ошибка компиляции
```

Упрощенно строка внутри похожа на структуру:

```
type stringHeader struct {
    data *byte
    len int
}
```

То есть строка хранит указатель на массив байтов и длину в байтах.

Для перевода массива байтов и массива рун в строку используется функция `string()` :

```
str := "hello мир"

bytes := []byte(str)
runes := []rune(str)

fmt.Println(string(bytes)) // hello мир
fmt.Println(string(runes)) // hello мир
```

Длина строки

`len(s)` - возвращает длину строки в байтах, а не количество символов.

```
package main

import "fmt"

func main() {
    sl := "helloo"
    sk := "привет"

    fmt.Printf("%s : %d\n%s : %d\n", sl, len(sl), sk, len(sk))
}
// ❯ go run string_ex.go
// helloo : 6
// привет : 12
```

Индексация строки

Если обратиться к элементу строки по индексу, то получим байт в массиве байтов, а не символ:

```
package main

import "fmt"

func main() {
    sl := "helloo"
    sk := "привет"

    fmt.Printf("%s : %d\n", sl, len(sl))

    fmt.Printf("str: %s\nbyte: %d\nsymbol: %c\n\n",
        string(sl[0]),
        sl[0],
```

```
        sl[0])

    fmt.Printf("%s : %d\n", sk, len(sk))
    fmt.Printf("str: %s\nbyte: %d\nsymbol: %c\n2 bytes: %s\n\n",
        string(sk[0]),
        sk[0],
        sk[0],
        sk[0:2])
}
// ❯ go run string_ex.go
// helloo : 6
// str: h
// byte: 104
// symbol: h

// привет : 12
// str: Ð
// byte: 208
// symbol: Ð
// 2 bytes: п
```

Мы получили непонятно что для кириллицы, потому что `sk[0]` это первый байт буквы `п`, а сама буква `п` занимает 2 байта.

Но когда мы сделали срез из двух байт, то получили одну букву `п`.

То есть так нельзя нормально работать с Unicode символами.

Rune

`rune` - alias для `int32`, Unicode code point (уникальное число, в которое UTF-8 кодирует символ).

То есть одна руна обычно соответствует одному Unicode символу в смысле code point (но бывают символы типа `é`, которые представляют из себя две руны).

```
package main

import "fmt"

func main() {
    r := 'a'  // латиница
    rs := "a" // латиница

    k := 'a'  // кириллица
    ks := "a" // кириллица

    fmt.Printf("rune: %d, type: %T\n", r, r)
    fmt.Printf("byte: %d, type: %T\n", rs[0], rs)
```

```
    fmt.Printf("rune: %d, type: %T\n", k, k)
    fmt.Printf("byte1: %d, byte2: %d, type: %T\n", ks[0], ks[1], ks)
}
// ❯ go run string_ex.go
// rune: 97, type: int32
// byte: 97, type: string
// rune: 1072, type: int32
// byte1: 208, byte2: 176, type: string
```

Одинарные кавычки в Go создают `rune literal`, а двойные кавычки в Go создают `string literal`.

```
r := 'a' // rune
s := "a" // string
```

Еще пример:

```
package main

import "fmt"

func main() {
    samples := []string{
        "a",
        "п",
        "世",
        "😊",
    }

    for _, s := range samples {
        fmt.Printf("%q: len(bytes)=%d bytes=%v runes=%v\n",
            s, len(s), []byte(s), []rune(s))
    }
}
// ❯ go run string_ex.go
// "a": len(bytes)=1 bytes=[97] runes=[97] кт
// "п": len(bytes)=2 bytes=[208 191] runes=[1087]
// "世": len(bytes)=3 bytes=[228 184 150] runes=[19990]
// "😊": len(bytes)=4 bytes=[240 159 152 138] runes=[128522]
```

Для подсчета количества символов в строке используется `utf8.RuneCountInString` или `len([]runte(str))`:

```
package main

import (
    "fmt"
```

```
    "unicode/utf8"
)

func main() {
    str := "привет world 😊 a"

    fmt.Printf("len bytes: %d\n", len(str))
    fmt.Printf("len runes: %d\n", utf8.RuneCountInString(str))
    fmt.Printf("len runes: %d\n", len([]rune(str)))
}
// ❯ go run .
// len bytes: 25
// len runes: 16
// len runes: 16

// п р и в е т _ w o r l d _ 😊 _ a
// 2 2 2 2 2 2 1 1 1 1 1 1 1  4  1 1 [кол-во байт]
// итого 16 символов и 25 байт:
// 6 символов кириллицей = 6 * 2 = 12 байт
// 1 эмоджи = 4 * 1 = 4 байта
// 9 символов латиницей = 9 * 1 = 9 байт
// 9 + 4 + 12 = 25 байт
```

Range по строке

Если использовать `range` по строке, то он будет идти по рунам, а индекс `i` - байтовый индекс начала руны.

```
package main

import "fmt"

func main() {
    str := "привет world 😊 a"

    for i, r := range str {
        fmt.Printf("byte index: %d, rune: %c\n", i, r)
    }

    fmt.Printf("len bytes: %d\n", len(str))
    fmt.Printf("len runes: %d\n", utf8.RuneCountInString(str))
}
// ❯ go run string_ex.go
// byte index: 0, rune: п
// byte index: 2, rune: р
// byte index: 4, rune: и
// byte index: 6, rune: в
// byte index: 8, rune: е
// byte index: 10, rune: т
```

```
// byte index: 12, rune:
// byte index: 13, rune: w
// byte index: 14, rune: o
// byte index: 15, rune: r
// byte index: 16, rune: l
// byte index: 17, rune: d
// byte index: 18, rune:
// byte index: 19, rune: 😊
// byte index: 23, rune:
// byte index: 24, rune: a
// len bytes: 25
// len runes: 16
```

В UTF-8 первый байт символа подсказывает, сколько всего байт занимает руна.

Упрощенно:

```
0xxxxxxx                -> 1 байт
110xxxxx 10xxxxxx       -> 2 байта
1110xxxx 10xxxxxx ...   -> 3 байта
11110xxx 10xxxxxx ...   -> 4 байта
```

То есть range делает примерно так:

1. смотрит на байт с индексом i
2. по первому байту понимает размер UTF-8 символа
3. декодирует rune
4. возвращает i и rune
5. увеличивает i на размер этой rune в байтах

Изменение строк

Так как строки *immutable*, то Go не позволяет менять участок памяти строки даже через `[]rune` или `[]byte`.

Поэтому когда мы преобразуем строку в `[]byte`, `[]rune` или обратно, то создается новая копия строки.

```
package main

import (
    "fmt"
)

func main() {
    str := "abcde"
```

```
    bytes := []byte(str) // тут создается копия

    bytes[0] = 120 // x

    fmt.Println(string(bytes)) // и тут создается копия
    fmt.Println(str)

    bytes[0] = 'X'

    fmt.Println(string(bytes)) // и опять создается копия
    fmt.Println(str)

    runes := []rune(str) // тут создается копия
    runes[0] = 0x41f     // п

    fmt.Println(string(runes)) // и тут создается копия
    fmt.Println(str)

    runes[0] = 'X'
    fmt.Println(string(runes)) // и опять создается копия
    fmt.Println(str)
}
// ❯ go run .
// xbcde
// abcde
// Xbcde
// abcde
// Пbcde
// abcde
// Xbcde
// abcde
```

Срез строки

Срез строки выполняется по байтовым индексам.

То есть это скорее всего будет нормально работать только если в строке одни ASCII символы.

Для Unicode символов уже будет опасно, потому что один символ может занимать >1 байта.

Тогда лучше преобразовывать в слайс рун и резать его, потом обратно преобразовывать в string.

```
package main

import (
    "fmt"
)
```

```go
func main() {
    strK := "абвгде"
    strL := "abcdef"

    fmt.Printf("re-slized strK[1:3]: %s\n", strK[1:3])
    fmt.Printf("re-slized strL[1:3]: %s\n", strL[1:3])

    fmt.Println()

    fmt.Printf("re-slized strK[2:6]: %s\n", strK[2:6])
    fmt.Printf("re-slized strL[2:6]: %s\n", strL[2:6])

    strKBytes := []byte(strK)
    strKRunes := []rune(strK)

    fmt.Printf("strKBytes[1:3]: %s\n", string(strKBytes[1:3]))
    fmt.Printf("strKBytes[2:6]: %s\n", string(strKBytes[2:6]))

    fmt.Printf("strKRunes[1:3]: %s\n", string(strKRunes[1:3]))
    fmt.Printf("strKRunes[2:6]: %s\n", string(strKRunes[2:6]))
}
// ❯ go run .
// re-slized strK[1:3]: ��
// re-slized strL[1:3]: bc

// re-slized strK[2:6]: бв
// re-slized strL[2:6]: cdef
// strKBytes[1:3]: ��
// strKBytes[2:6]: бв
// strKRunes[1:3]: бв
// strKRunes[2:6]: вгде
```

Если реслайзнем так, что половина байт символа отвалится, то выведется символ `�`

Это специальный символ для некорректных UTF-8 символов.

Для среза строки лучше использовать руны.

Строковые литералы

В Go есть два основных вида строковых литералов:

- Interpreted string literal `""`
- Raw string literal ``

Interpreted string literal

Interpreted потому что Go интерпретирует специальные последовательности внутри строки:

```
fmt.Println("hello\nworld")
// hello
// world
```

Raw string literal

Raw потому что они не интерпретируются, а берется как есть:

```
fmt.Println(`hello\nworld`)
// hello\nworld
```

Raw может быть многострочной, поэтому удобна для `SQL`, `regex`, `JSON`, многострочного текста и т.д.

Сравнение строк

Строки сравнимы в лексикографическом порядке по байтам.

```
fmt.Println("abc" < "def")
// true
```

Пакет strings

Для работы с строками используется пакет `strings`.

strings.Contains

```
strings.Contains("hello world", "world") // true
```

strings.HasPrefix

```
strings.HasPrefix("Bearer token", "Bearer ") // true
```

strings.HasSuffix

```
strings.HasSuffix("file.txt", ".txt") // true
```

strings.TrimSpace

```
s := "  hello \n"
fmt.Println(strings.TrimSpace(s)) // hello
```

strings.Trim

```
fmt.Println(strings.Trim("!!!hello!!!", "!")) // hello
```

strings.ToLower, strings.ToUpper

```
fmt.Println(strings.ToLower("GoLang")) // golang
fmt.Println(strings.ToUpper("GoLang")) // GOLANG
```

strings.Split

```
parts := strings.Split("a,b,c", ",")
fmt.Println(parts) // [a b c]
```

strings.Join

```
parts := []string{"a", "b", "c"}
s := strings.Join(parts, ",")

fmt.Println(s) // a,b,c
```

strings.Replace

```
s := strings.Replace("go go go", "go", "GO", 2)
fmt.Println(s) // GO GO go
```

Последний аргумент — сколько замен делать.

strings.Replace(s, old, new, n)

Если n = -1, заменить все:

```
s := strings.Replace("go go go", "go", "GO", -1)

fmt.Println(s) // GO GO GO
```

strings.ReplaceAll

```
s := strings.ReplaceAll("go go go", "go", "GO")
fmt.Println(s) // GO GO GO
```

strings.Fields

Разбивает по whitespace:

```
s := " hello   world \n go "
fields := strings.Fields(s)

fmt.Println(fields) // [hello world go]
```

strings.Repeat

```
fmt.Println(strings.Repeat("-", 10)) // ----------
```

strings.Index

```
fmt.Println(strings.Index("hello", "ll")) // 2
fmt.Println(strings.Index("hello", "x"))  // -1
```

Возвращает байтовый индекс.

Самые частые функции

Вообще для работы со строками чаще всего используют пакеты:

- strings
- strconv
- unicode/utf8

Самые частые методы:

```
utf8.RuneCountInString(s)
utf8.ValidString(s)

unicode.IsLetter(r)
unicode.IsDigit(r)
unicode.IsSpace(r)
unicode.ToUpper(r)
unicode.ToLower(r)

strconv.Itoa(i)
```

```
strconv.Atoi(s)
strconv.FormatInt(i, 10)
strconv.ParseInt(s, 10, 64)
strconv.FormatFloat(f, 'f', 2, 64)
strconv.ParseFloat(s, 64)

b.WriteString(s)
b.WriteByte(',')
b.WriteRune('п')
b.String()
b.Grow(n)
b.Reset()
```

String Builder

Для эффективной сборки строки по частям используется `strings.Builder`.

Потому что при конкатенации строк происходит создание новой строки:

```
s := "hello"
s = s + " world" // тут создается новая строка
```

Используется так:

```
package main

import (
    "fmt"
    "strings"
)

func main() {
    var b strings.Builder
    defer b.Reset()

    b.WriteString("Hello")
    b.WriteByte(' ')
    b.WriteString("Vlad")
    b.WriteRune('!')
    b.WriteByte('\n')
    b.WriteString("Привет")

    result := b.String()

    fmt.Println(result)
}
// ❯ go run .
```

```
// Hello Vlad!
// Привет
```

ВАЖНО!!!

Нельзя копировать `strings.Builder` после первого использования, если нужно, то передавать pointer на него:

```
func writeName(b *strings.Builder, name string) {
    b.WriteString(name)
}
```

IO